Journal of Software Engineering Research and Development
This work is licensed under a Creative Commons Attribution 4.0 International License.

HalluCodeDetection: Detecting Hallucinations in Code Generation

Rubens Nascimento de Lima

Abstract Small Language Models (SLMs) have gained increasing attention for code-generation tasks because their reduced computational requirements enable local execution, offering practical advantages in deployment cost, privacy, and data control. However, code generation remains susceptible to hallucinations, and this limitation may be particularly relevant for smaller models operating with more constrained capacity. More importantly, producing code and recognizing when generated code is incorrect are distinct capabilities that remain insufficiently understood for SLMs. In this work, we investigate whether SLMs can act not only as code generators but also as hallucination detectors, whether specialized training can improve this capability, and how their predictions compare with and benefit from deterministic static-analysis tools. We evaluate these questions on all 563 HumanEval+ and MBPP+ tasks from EvalPlus, using execution-based labels for Syntax, Runtime, Functional, and Correct outcomes. SLMs generate correct solutions for up to 80.8% of the tasks, reaching the same performance range as the substantially larger models evaluated in our comparison. Their predominant failures are not malformed syntax or execution crashes, but Functional hallucinations: programs that execute while deviating from the requested behavior. Error detection, however, follows a different pattern. No general-purpose model performs consistently well across all error categories, and Runtime hallucinations are particularly difficult, with the highest observed Runtime Recall reaching only 32.9%. Nevertheless, several SLMs detect more Runtime failures than Pyright, which identifies only 8.9% of the held-out Runtime cases under our rule mapping, showing that model-based reasoning can capture failures beyond those exposed by the selected static analysis. The two approaches are complementary: compilation provides deterministic Syntax detection, while incorporating tool feedback raises Syntax Recall to nearly 100% for most models. However, its effect on Runtime and Functional judgments remains model-dependent. Finally, specialized training substantially improves hallucination detection: an Accuracy-oriented detector gains 32.0 percentage points over its base model, while Macro-F1-oriented training produces a more balanced detector across error categories. Overall, our results show that code generation, hallucination detection, and correction are distinct capabilities, and that combining specialized SLMs with deterministic software-engineering tools provides complementary evidence for identifying failures in generated code.

Keywords: *Code Generation, Hallucination Detection, Specialized Fine-tuning, Small Language Models*

1 Introduction

Small Language Models (SLMs) have gained increasing attention for code-generation tasks because their lower computational requirements make local deployment more feasible, reducing infrastructure costs while providing greater control over data and privacy. These characteristics make SLMs attractive for programming assistants and other software-engineering applications in which source code may be sensitive or computational resources are limited. However, their more restricted model capacity does not eliminate the reliability problems associated with generative models. Generated code may appear plausible while still containing defects or deviating from the requested behavior, a phenomenon commonly discussed as *code hallucination* (Pavel et al., 2025; Lee et al., 2025). As SLMs become increasingly capable code generators, understanding whether they can also recognize problems in generated code becomes important for assessing their reliability beyond generation performance alone.

Detecting these problems is difficult because a hallucination and its observable consequence are not necessarily the same concept. In this work, we use *hallucination* to describe the generative phenomenon that leads a model to produce plausible but incorrect code, while the resulting failures are operationalized as observable error outcomes. Following prior work on code-generation failures (Lee et al., 2025; Tian et al., 2025), we distinguish *Syntax Error*, *Runtime Error*, and *Functional Error*, together with *Correct* solutions. Syntax Errors can often be identified mechanically, whereas Runtime Errors may emerge only under particular execution conditions, and Functional Errors may remain hidden because the program executes normally while producing behavior inconsistent with the specification. Test execution can provide objective evidence when sufficiently comprehensive tests are available, but such information is not necessarily accessible to a model during code assessment. Moreover, generating correct code and recognizing errors are different tasks, so strong generation performance does not necessarily imply strong evaluation capability.

Our central idea is therefore to study SLMs beyond their role as code generators and evaluate whether they can also serve as code-error detectors. We investigate whether general-purpose SLMs can recognize different categories of errors, whether they can identify errors in code they generated themselves, and whether task specialization can improve this capability without increasing model scale. We further compare model-based detection with deterministic static-analysis techniques to understand which types of errors each approach can identify and whether their signals are complementary. Finally, we examine whether external static-analysis evidence can help SLMs both classify gener-

ated code and revise their own solutions, and we compare these behaviors with those observed in substantially larger language models.

To operationalize this investigation, we use all 563 programming tasks from HumanEval+ and MBPP+ provided by EvalPlus (Liu et al., 2023). Multiple SLMs first generate solutions, which are compiled and executed against the EvalPlus test suites to derive an execution-based reference indicating *Syntax Error*, *Runtime Error*, *Functional Error*, or *Correct*. Runtime and Functional Errors may co-occur when different test cases expose different behaviors. The generated programs are then presented to language models without test results or reference labels, turning error detection into an independent multilabel classification task. The study is organized around five questions: how well SLMs generate correct code; how well they detect errors and evaluate their own outputs; whether specialized SLMs improve error detection; how static-analysis techniques complement SLMs in detection and iterative generation; and how these behaviors compare with substantially larger language models. For specialization, we construct supervised training data from execution-derived labels and teacher-generated explanations and fine-tune Gemma-based models. Static-analysis signals are evaluated both independently and as additional evidence supplied to the models.

The results reveal a clear separation between generation and error detection. The evaluated SLMs generate correct solutions for up to 80.8% of the EvalPlus tasks, with several models reaching the same observed performance range as the two substantially larger models used for comparison. Their predominant failures are Functional Errors rather than Syntax Errors, showing that the main difficulty is often not producing executable Python code, but producing code that fully satisfies the requested behavior. Error detection is substantially more heterogeneous. No general-purpose model performs consistently well across all categories, and Runtime Errors are particularly difficult, with the highest observed Runtime Recall reaching only 32.9%. Even under this limitation, some SLMs identify more Runtime Errors than the static-analysis technique evaluated for this category, which detects only 8.9% of the held-out Runtime cases. Static analysis is considerably stronger for syntactic failures, and providing this evidence to the models raises Syntax Error detection to nearly perfect levels for most evaluated models, although its effects on Runtime and Functional decisions remain model-dependent. Specialization further improves detection: the Accuracy-oriented detector gains 32.0 percentage points over its base model, while Macro-F1-oriented training produces a more balanced profile across error categories. Together, these results indicate that generation, error recognition, interpretation of external evidence, and code revision represent related but distinct capabilities.

The main contributions of this work are:

- An empirical evaluation of SLMs for both code generation and error detection under a common execution-based reference, distinguishing Syntax, Runtime, and Functional Errors through multilabel classification.
- An analysis of *self-error detection*, investigating whether models can recognize errors in code they generated themselves and examining the relationship between generation and evaluation capabilities.
- A specialized SLM-based detector trained with execution-derived labels and teacher-generated explanations, together with an analysis of how different optimization objectives affect the detector's behavior across error categories.
- An investigation of the relationship between model-based reasoning and deterministic static analysis, evaluating static techniques independently, as supporting evidence for error detection, and as feedback during iterative code generation.
- A comparison between SLMs and substantially larger language models under the same generation, detection, and tool-assisted protocols, showing that selected SLMs can occupy the same observed performance range while requiring substantially fewer parameters.

The remainder of this paper is organized as follows. Section 2 reviews previous work on code hallucinations, error characterization, automatic evaluation, specialized detectors, and feedback-guided code generation. Section 3 describes the experimental design, evaluated models, execution-based reference construction, specialized training, and integration of static-analysis techniques. The results section addresses the five research questions concerning generation, error detection, specialization, static-analysis assistance, and comparison with substantially larger models. The paper then discusses threats to validity and concludes by summarizing the main findings and outlining directions for future work.

2 Related Work

2.1 Hallucinations and error taxonomies in code generation

The term *hallucination* has no single, universally accepted operational definition in code generation. Pavel et al. (2025) characterize hallucinations as outputs that appear plausible but are factually incorrect, unverifiable, or nonsensical, and report manifestations such as syntax violations, invalid references, and conflicts with API knowledge. This perspective is particularly relevant to code generation because syntactically plausible source code may still fail during execution or behave differently from the requested specification.

Different studies have proposed taxonomies to characterize these failures. Lee et al. (2025) organize code-generation hallucinations into syntactic, runtime-execution, functional-correctness, and code-quality categories. Dou et al. (2025) provide another important perspective by organizing failures in generated code into three broad bug categories and ten subcategories. Their taxonomy was an important motivation for the classification adopted in this work because it reinforces that generated-code failures should not be treated as a single undifferentiated notion of incorrectness. CodeHalu provides a different decomposition, grouping hallucinations into mapping, naming, resource, and logical categories through execution-based verification (Tian et al., 2025).

These taxonomies characterize generated-code failures from different perspectives, including their source, manifes-

tation, and observable effect. In this work, we distinguish the hallucination phenomenon from the error produced as its observable consequence. We do not attempt to infer the underlying cause of a hallucination from the generated program. Instead, we operationalize its effects as *Syntax Error*, *Runtime Error*, and *Functional Error*, together with *Correct* for solutions that satisfy the available evaluation suite. This outcome-oriented abstraction is intentionally coarser than some previous taxonomies, but it enables each category to be grounded consistently in compilation and execution behavior. Code-quality issues are outside the scope of this study because our focus is correctness and executability rather than style, maintainability, or efficiency.

2.2 Empirical evidence on failures in generated code

Empirical studies show that failures in generated code extend well beyond compilation problems. Dou et al. (2025) evaluate closed- and open-source models across multiple benchmarks and show that error distributions depend on both the evaluated model and the characteristics of the programming tasks. Their results also indicate that real-world tasks can exhibit failure distributions different from those observed in conventional benchmarks. Similarly, Pavel et al. (2025) show that generated-code failures can depend on domain knowledge and API constraints, even when additional contextual information is provided to the model.

These findings motivate analyzing error categories explicitly rather than relying only on an aggregate correctness measure. However, characterizing the errors produced by a generator does not determine whether a model can recognize those same errors when examining generated code. This distinction is central to our study. We evaluate generation and error detection under the same execution-based reference to investigate whether models that generate code successfully are also capable of distinguishing Correct solutions from Syntax, Runtime, and Functional Errors. We further examine this relationship for self-evaluation, where a model is asked to identify errors in code that it generated itself.

2.3 Automatic evaluation and error detection

Execution against tests provides a direct mechanism for identifying many observable failures in generated programs. EvalPlus strengthens HumanEval and MBPP with additional automatically generated tests and demonstrates that the original test suites can accept incorrect solutions and affect the relative assessment of code-generation models (Liu et al., 2023). This motivates the use of strengthened execution-based evaluation when constructing reference labels. In our study, compilation and EvalPlus execution determine the reference error outcomes, while the language models evaluated as detectors do not receive these execution results.

CodeHalu follows a related execution-based direction and introduces CodeHaluEval, containing 8,883 generated programs from 699 tasks (Tian et al., 2025). Its objective, however, differs from ours in both taxonomy and detection procedure. CodeHalu characterizes hallucinations using its own error taxonomy and an execution-based heuristic pipeline, whereas HalluCodeDetection formulates error detection as a multilabel classification problem over observable Syntax, Runtime, Functional, and Correct outcomes.

The detector-oriented work most closely related to ours is Andriushchenko et al. (2026), who construct line-level hallucination annotations and train a lightweight Transformer detector using internal representations from language models. Their results demonstrate that specialized detectors can identify hallucinated regions and outperform generic uncertainty-based approaches. Our work investigates a different granularity and setting. Rather than locating individual hallucinated lines, we ask whether general-purpose and specialized SLMs can classify the observable failure mode of a complete generated solution. We additionally study self-error detection, class-specific behavior, and the interaction between model-based detection and external software-analysis evidence.

2.4 Specialized evaluators and rationale-based supervision

Learned evaluators have previously been incorporated into code-generation systems. CodeRL trains a critic to estimate functional correctness and uses critic predictions together with unit-test feedback to improve a code generator through reinforcement learning (Le et al., 2022). Its outcome space includes compilation errors, runtime errors, failed tests, and passed tests, making it an important precedent for distinguishing different observable execution outcomes. However, the critic in CodeRL is primarily a component used to optimize generation rather than a stand-alone multilabel detector evaluated across different error categories.

Our specialized detector instead treats error recognition as the primary task. We fine-tune SLMs to predict the reference error categories directly and investigate how optimization objectives affect the resulting decision behavior. The detector is trained not only from execution-derived labels but also from natural-language explanations produced by teacher models. This design follows previous work showing that rationale-based supervision can provide richer training signals than labels alone. **?** use generated rationales as additional supervision when distilling knowledge into smaller models, while **?** study explanation-based fine-tuning as a mechanism for reducing reliance on spurious cues. In HalluCodeDetection, the teacher-generated explanations serve only as auxiliary supervision: the reference labels remain determined by execution, and detector performance is evaluated exclusively from the predicted error classes.

The distinction from previous critic-based systems is therefore not the general idea of using a model to assess generated code. Rather, we investigate whether an SLM can be specialized as an independent multilabel error detector and whether specialization changes its ability to distinguish individual error categories, particularly under a strongly imbalanced class distribution.

2.5 Self-evaluation, self-correction, and tool-assisted generation

Several approaches improve model outputs through iterative feedback. Self-Refine alternates between generation, self-

feedback, and refinement without additional model training (Madaan et al., 2023), while Reflexion uses verbal feedback from previous attempts to guide subsequent decisions and includes programming tasks among its applications (Shinn et al., 2023). These approaches demonstrate that models can sometimes use their own assessments to revise an output, but successful revision does not necessarily imply that the underlying error was identified correctly.

For code generation, Self-Debugging asks a model to inspect execution information and reason about its program before producing a revised solution (Chen et al., 2024). Dou et al. (2025) similarly investigate a training-free correction process based on self-critique, bug information, and compiler feedback. These studies establish that external evidence and iterative revision can improve code-generation workflows. Our study separates three related capabilities that are often considered together: recognizing an error, interpreting external evidence about that error, and successfully revising the program.

This distinction also motivates our comparison with deterministic software-analysis techniques. Compilers and static analyzers can provide direct evidence for some classes of errors, while language models may reason about behaviors that are not captured by a particular static analysis. We therefore evaluate static techniques independently, use their outputs as supporting evidence for model-based classification, and provide the same evidence during iterative code generation. This allows us to examine not only whether an error signal exists, but also whether an SLM can interpret and act on that signal appropriately.

Taken together, prior work has established taxonomies for code hallucinations, execution-based evaluation, specialized critics, self-correction, and tool-assisted generation. HalluCodeDetection builds on these directions but studies a combination that remains less explored for SLMs: generation and error detection under the same execution-based reference, self-error detection, specialization as a stand-alone multilabel detector, and the complementarity between learned detection and deterministic software-analysis evidence. This design allows us to investigate whether generation, error recognition, evidence interpretation, and correction behave as the same capability or as distinct aspects of reliable code generation.

3 Methodology

3.1 Study Overview

This study investigates the ability of Small Language Models (SLMs) to both generate correct code and identify errors in code generated by other models. The experiment combines code generation, automated classification, knowledge distillation, and specialized training to evaluate different dimensions of the problem.

Several language models generate solutions for standardized programming tasks. Each solution is executed against a test suite that objectively determines whether the code is correct or contains syntax, runtime, or functional errors. This ground truth classification is used both to evaluate the code generation capability of the models and as the basis for detection experiments.

The study includes a knowledge distillation process in which language models act as judges to generate natural language explanations for each identified error type. These explanations are associated with the reference labels and compose the data used for supervised training of a specialized error detection model via *fine-tuning* with the QLoRA technique.

Detection capability is evaluated for both general-purpose models, without specific training, and the specialized model. In both cases, the model receives only the programming task description and the generated code, without access to reference labels or test execution results.

The detection task is treated as multilabel classification, since the same code may contain more than one type of error simultaneously.

The experimental factors are the evaluated model, the specialization/training strategy, and the availability of static-analysis feedback. Direct comparisons keep the programming tasks, taxonomy, prompt structure, decoding temperature, output schema, metrics, and evaluation harness fixed. Model family and pretraining data, however, necessarily vary with the model, and backend, numerical precision, quantization, and hosted-model revision are not fully controlled. Consequently, cross-model findings are interpreted as benchmark-conditioned associations rather than causal effects of parameter count.

3.2 Evaluated Models

The study evaluates language models with parameter counts ranging from 4B to 552B, as shown in Table 1. The table separates the small language models that are the main focus of the study from the two large models used only for the comparison with larger models.

The local SLMs used in the initial code generation were executed with Q4_K_M quantization, which allowed larger models to be evaluated within the available computational budget. The Gemma models were also evaluated without precision reduction in the detection experiments. The fine-tuned models were executed without inference quantization; their adaptation used QLoRA with 4-bit NF4 quantization (Dettmers et al., 2023), which is a training mechanism distinct from Q4_K_M inference quantization. The two large models were accessed through a hosted API, so no quantization level is assumed for them.

For the knowledge distillation process, three models were selected as judges: Muse Glimmer 30B, Qwen 3.8 27B, and Ornith 1.5 35B. These models were chosen because they are the largest general-purpose models and demonstrated strong performance in the code generation step.

For the specialized model training, the Gemma 4 E2B and E4B models were used as base models for *fine-tuning*, along with Gemma 3 4B.

During the knowledge distillation and error detection evaluation, all models were configured with a temperature of 0.0 to ensure deterministic outputs. Across all experiments, both the SLMs and the larger comparison models were executed in direct-response mode, with thinking or reasoning mode and

Table 1. Models evaluated in the study

Model	Parameters
Small language models	
Devstral Small 2	24B (Rastogi et al., 2025) (Mistral AI, 2025)
GLM 4.7 Flash	30B (GLM-4.5 Team et al., 2025) (Z.ai, 2025)
Gemma 3 4B	4B (Gemma Team et al., 2025)
Gemma 4	{5.1, 8, 31}B (Gemma Team et al., 2026)
GPT-OSS 20B	20B (OpenAI et al., 2025)
Muse Glimmer 30B	30B (Meta Superintelligence Lab, 2026)
Ornith 1.5 35B	35B (Ornith Team, 2026)
Qwen 2.5 Coder	{7, 32}B (Hui et al., 2024)
Qwen 3 Coder 30B	30B (Yang et al., 2025) (Qwen Team, 2025)
Qwen 3.8 27B	27B (Qwen Team, 2026)
Large models (comparison)	
DeepSeek V4.1 Flash	552B (DeepSeek-AI, 2026)
MiMo V2.5	310B (Xiaomi MiMo Team, 2026)

reasoning effort disabled whenever these controls were exposed by the model backend. Consequently, the models were not prompted to produce a chain-of-thought or a separate reasoning trace, and no such trace was retained or supplied as input to any subsequent phase.

3.3 Programming Tasks and Code Generation

The experiments utilize the EvalPlus benchmark (Liu et al., 2023), which extends both HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) with additional test cases to increase the reliability of correctness assessment. EvalPlus provides 164 tasks from HumanEval and 399 tasks from MBPP, totaling 563 programming tasks.

We use all 563 tasks available in these two EvalPlus portions rather than selecting a random subset. This complete benchmark coverage avoids sampling tasks after observing model behavior, but it does not make the tasks representative of all programming domains. HumanEval+ and MBPP+ are appropriate here because they provide executable Python function-level problems and strengthened tests under a common harness; at the same time, benchmark, language, task-length, difficulty-distribution, prompting, model-family, and tool effects remain potentially confounded.

Each task consists of a natural language problem description that specifies the required functionality. The problems cover diverse algorithmic challenges, including string manipulation, mathematical operations, and data structure operations. For HumanEval tasks, the model receives the function signature and must complete the function body. For MBPP tasks, the model generates the complete function implementation.

The code generation process uses a system prompt that instructs the model to act as a Python assistant that solves coding problems, returning only a single function implementation without tests, asserts, or example usage. The user prompt varies according to the benchmark: for HumanEval, it requests the completion of a function body; for MBPP, it requests the complete function implementation.

For the initial code generation, each model produced one solution per programming task, using a temperature of 1.0 to allow for diversity. The top-p and top-k parameters were kept at the defaults of each model or backend, and no fixed random seed was imposed.

The raw response is normalized before execution. The extraction step removes Markdown code fences when they wrap the whole response and detects whether the model already returned the entry-point function definition. For HumanEval completion tasks, the harness reuses the task prompt, which already contains the function header, and appends the normalized function body; when the model returned the complete function definition, that definition is used directly. For MBPP, the returned function definition is used when present; otherwise the required signature is prepended and the body is indented. This normalization does not remove arbitrary natural-language text that the model may produce before or after the code, so a response that mixes prose with code can still fail to compile. When extraction produces an incomplete source, for example when the function header is lost, the ground-truth compilation classifies the solution as Syntax.

3.4 Ground Truth and Error Classification

The ground truth classification is determined by executing each generated solution against a custom test harness that evaluates the code at different levels of correctness. Labels are accumulated across the test cases, so a single solution may receive more than one label, with the exception of Correct.

The procedure starts with a compilation step. If the source cannot be compiled, it receives the Syntax label and the evaluation stops, so Syntax is exclusive and no other label is added. It is important to note that syntax errors may arise from different sources: (1) genuine Python syntax errors in the generated code, such as invalid import statements or malformed expressions; (2) formatting issues, such as the model returning natural language explanations or Markdown fences along with the code; or (3) extraction issues, such as the loss of the function header during code processing.

If the source compiles, it is executed in an isolated subprocess against the base test cases and the additional test cases. For each test case, if the execution raises an exception, the Runtime label is added; if the execution completes but returns a result different from the expected one, the Functional label is added. Runtime and Functional can therefore co-occur in the same solution, because some test cases may raise exceptions while others return incorrect results. A timeout, or a failure that prevents the execution of the candidate as a whole, is also labeled as Runtime.

The Correct label is assigned only when the source compiles, no test case raises an exception, and all test cases re-

turn the expected result. It is therefore exclusive: a solution labeled Correct cannot receive any other label, and a solution that receives any error label cannot receive Correct. The procedure is summarized in Figure 1.

```
Input: generated code, EvalPlus tests
labels = {}
if code does not compile:
    labels += Syntax
    return labels
for each test case:
    execute code
    if exception or timeout:
        labels += Runtime
        continue
    if output != expected result:
        labels += Functional
if labels is empty:
    labels += Correct
return labels
```

Figure 1. Ground truth labeling procedure. Runtime and Functional accumulate across test cases, whereas Syntax terminates the evaluation and Correct is assigned only when no other label is present.

3.5 Error Detection Experiment

The error detection experiment evaluates whether language models can correctly identify errors in generated code without access to reference labels or test execution results. For each sample, the model receives only the programming task description and the generated code.

The models are prompted with a system prompt that defines the classification task. Table 2 describes the four error categories used in this study.

Table 2. Error categories and their definitions

Error Type	Definition
Syntax	When the provided code has a syntax error in the Python language, such as incorrect or incomplete code.
Runtime	When the code is "compilable" but has an error when executed, such as a call to an invalid function, an undefined variable, etc.
Functional	When the code is executable, but it has some deviation from the correct functioning of what was requested.
Correct	When the code is well-formed and has no functional errors.

The prompt instructs the model to return a JSON object containing a list of detected error types, each with a natural language explanation. The prompt emphasizes that the model should provide general explanations without mentioning specific test cases or input values.

All models evaluated in the code generation step are also evaluated in the error detection experiment. The models are evaluated under the same conditions, with a temperature of 0.0 to ensure deterministic outputs. The two large models used in the comparison with larger models follow exactly the same detection protocol as the SLMs: same set of examples, same task description, same input code, same system prompt, same class definitions, same output format, same metrics, and the same temperature. No additional information, tests, or ground truth is provided to them.

The Exact-set Accuracy measures the proportion of samples where the set of predicted error types exactly matches the set of ground truth error types. Additionally, Macro F1 and Macro Recall are computed by isolating each error class independently, providing a per-class analysis of detection capability.

3.6 Specialized Model Training and Knowledge Distillation

This section describes both the knowledge distillation process used to generate training data and the specialized model training procedure.

3.6.1 Knowledge Distillation

The knowledge distillation process generates natural language explanations for each error type identified in the ground truth classification. For each generated code sample, a judge model receives the problem description, the generated code, the name of the model that generated the code, and the ground truth error labels. The judge is instructed to write a general explanation for each error type present, describing why the code can fail in that way without mentioning specific test cases or input values. The judge does not assign or correct the label; it explains a label that was already determined by execution.

Every example is processed by all three judges: Muse Glimmer 30B, Qwen 3.8 27B, and Ornith 1.5 35B. These models were chosen because they are the largest general-purpose models and demonstrated strong performance in the code generation step. Each judge produces its own rationale for the same code and the same reference label, and no voting or winner selection is performed. Each generated sample therefore contributes one training record per judge, that is, three records that share the code and the label but carry a different rationale. Each rationale associates an error type with the explanation produced by the judge. If a judge fails to produce a response for a sample, no record is written for that judge and the sample is retried in a later execution. The reference label always comes from the execution of the code, and the rationale is only the target text that accompanies it.

Training the specialized model on the label together with an explanation, rather than on the label alone, provides richer auxiliary supervision. Label-only supervision indicates which output is expected but offers little explicit information about the relation between the task requirement, the behavior of the code, and the error category, which in an imbalanced dataset can favor superficial correlations and shortcuts associated with the dominant class. Conditioning the target on an explanation encourages the model to associate its prediction with evidence in the code. This follows the rationale-based distillation of Hsieh et al. (2023), which

uses rationales as additional supervision, and the explanation-based fine-tuning of Ludan et al. (2023), which reports reduced reliance on spurious cues. The explanations provide richer auxiliary supervision and are not claimed to eliminate bias or to guarantee robustness. In the quantitative evaluation, only the predicted classes enter the metrics; the textual quality of the explanations is not scored.

3.6.2 Specialized Model Training

The training dataset is the output of the knowledge distillation process described above. Each record in this dataset contains the problem description, the generated code, and the corresponding error labels with explanations generated by the judge models.

The base models selected for *fine-tuning* are Gemma 4 E2B, Gemma 4 E4B, and Gemma 3 4B. Each training sample is formatted as a three-turn conversation. The system prompt describes the classification task and defines the four error categories. The user message contains the programming task description and the generated code. The assistant message contains the expected output: a JSON object with the detected error types and their explanations, formatted as a Markdown code block.

The training uses QLoRA (Quantized Low-Rank Adaptation) with 4-bit NF4 quantization (Dettmers et al., 2023), applying low-rank adapters as introduced by LoRA (Hu et al., 2022). Table 3 shows the hyperparameters explored during the search.

Table 3. Training hyperparameters

Parameter	Values
LoRA rank (r)	{8, 16, 32}
LoRA alpha (α)	{8, 16, 32}
LoRA dropout	{0.05, 0.1}
Learning rate	{5e-5, 1e-4}
Bias	{none, all, lora_only}
Epochs	{3, 4}

The dataset is split into training, validation, and test sets. The split is performed at the task level, so all code samples for a given programming task are assigned to the same split, preventing data leakage where different solutions for the same task appear across splits. The validation set comprises 15% of the data, and the test set comprises 20%.

Four training configurations were evaluated. The first two use the original class distribution, without removing Syntax and without explicit balancing. The first is Accuracy-oriented: it uses the standard causal language-modeling loss and selects the best model by validation Accuracy. The second is Macro-F1-oriented: it uses the label-weighted loss, which assigns a higher weight to the class-name tokens, and selects the best model by validation Macro-F1.

The other two configurations balance the training split only. Correct examples are reduced, Syntax examples are removed from training, and the remaining examples are downsampled to a runtime:functional:correct ratio of 1:1:1 or 1:3:3, without duplicating rows. Validation and test remain unchanged, so balancing cannot move a task between splits. Both balanced configurations use the label-weighted loss; the third selects the best model by validation Accuracy, and the fourth by validation Macro-F1.

Hyperparameter search was performed with Optuna (Akiba et al., 2019) using the Tree-structured Parzen Estimator (TPE) sampler with multivariate sampling and constant-liar. The search space covers the base model, LoRA rank and alpha, LoRA dropout, learning rate, number of epochs, and bias configuration, as listed in Table 3. For each search, the best configuration is the one that maximizes the respective selection metric on the validation split, and the corresponding merged adapter is retained.

3.7 Evaluation Metrics

The evaluation uses metrics appropriate for multilabel classification scenarios. For each of the four error categories (syntax, runtime, functional, and correct), true positives (TP), false positives (FP), and false negatives (FN) are computed independently.

The per-class recall is calculated as $TP/(TP+FN)$, measuring the proportion of actual positive cases correctly identified for each error type. The per-class precision is calculated as $TP/(TP + FP)$, measuring the proportion of predicted positive cases that are actually correct.

The Macro F1 score is computed as the average of the F1 scores across all four error categories, where each category's F1 score is $2 \times TP/(2 \times TP + FP + FN)$. This metric treats all error types equally, regardless of their frequency in the dataset.

The Macro Recall is computed as the average of the recall scores across all four error categories, providing a measure of the overall detection capability across error types.

The Exact-set Accuracy measures the proportion of samples where the set of predicted error types exactly matches the set of ground truth error types. This strict metric requires a complete match of the multilabel prediction, reflecting the practical requirement that a detection system must correctly identify all present error types.

3.8 Statistical Analysis

This study is research-question driven and exploratory; it does not formulate a priori confirmatory hypotheses. When a predefined RQ involves a comparison, we use two-sided inferential procedures to quantify evidence for the observed difference. The programming task is the independent unit because solutions and rationale-expanded evaluation rows from the same task share a specification and tests. Conditions are paired whenever they evaluate the same task or generated program. All intervals and tests are conditional on the finite EvalPlus benchmark and do not treat its 563 tasks as a random sample of all software.

For RQ1, Correct is reduced to a binary outcome for the same 563 tasks in every model condition. We report Wilson 95% confidence intervals (CIs), use Cochran's Q as the omnibus paired comparison, and, after a significant omnibus result, apply exact two-sided McNemar tests to all 78 model pairs. Holm correction controls this pairwise family, and effect sizes are paired differences in percentage points with per-

centile CIs from 10,000 task-level bootstrap samples. RQ2 is descriptive: detector metrics receive task-clustered bootstrap CIs, preserving all rationale-expanded rows within a task, and self-detection proportions receive Wilson CIs; no post-hoc detector ranking test is introduced.

For RQ3, each specialized detector is matched with the Gemma 4 E4B base on common held-out samples. We use a paired task-cluster bootstrap for metric differences and a two-sided task-level label-swap permutation test. Holm correction is applied to the eight predefined primary comparisons (four specialized models by Accuracy and Macro-F1); class-level metrics are descriptive. For RQ4, standalone tool recall receives Wilson CIs. Phase 6 compares Correct before and after iterative feedback with exact McNemar tests, paired bootstrap CIs, and median/interquartile range (IQR) for cycles. Phase 7 uses the same task-clustered bootstrap and paired permutation procedure as RQ3; its available model/metric comparisons form one Holm family. RQ5 synthesizes those estimates without a new scale test because model size is confounded with architecture, training, precision, and backend and only two large-model baselines are available.

All bootstrap and permutation procedures use 10,000 resamples and analysis seed 20260920. Bootstrap assumes that tasks are independent clusters; paired permutation additionally assumes exchangeability of whole task clusters under the null. A single generation per task/model does not estimate generation-to-generation variance.

3.9 Static Analysis Tools

Two static analysis tools are used in this study, each exclusive to one failure category. Python compilation (`py_compile`) is responsible for the syntax category, and Pyright is responsible for statically detectable runtime issues.

Python compilation, implemented with the language's built-in `compile` function (Python Software Foundation, 2026), parses the source code into a code object and reports a syntax error when the source cannot be parsed. A compilation failure is mapped to Syntax, whereas a successful compilation is interpreted as the absence of a syntax signal. It is never used to detect Runtime or Functional errors, and it does not execute the code. Because it only parses the source, it is fast and deterministic.

Pyright is a standards-based static type checker for Python (Microsoft, 2026) that analyzes the source code without executing it. It builds a model of the program and reports conditions that would fail at execution. In this study it is invoked under the default type checking mode with `pyright --outputjson --level error --pythonversion 3.13`, so only diagnostics with error severity are parsed. A diagnostic is mapped to Runtime only when its rule belongs to the runtime-related set considered here, which includes `reportUndefinedVariable`, `reportUnboundVariable`, `reportPossiblyUnboundVariable`, `reportCallIssue`, `reportArgumentType`, `reportAssignmentType`, `reportAttributeAccessIssue`, `reportIndexIssue`, `reportOperatorIssue`, `reportReturnType`, `reportOptionalCall`, `reportOptionalMemberAccess`, `reportOptionalOperand`, `reportOptionalSubscript`, `reportOptionalIterable`, `reportOptionalContextManager`, `reportOptionalEnter`, `reportGeneralTypeIssues`, and `reportInvalidTypeArguments`. Diagnostics without a rule whose message indicates a parsing problem, such as an expected token or an indentation error, are mapped to Syntax. All other diagnostics, including style and type-annotation warnings, are ignored. Pyright detects runtime issues that compilation cannot, but it cannot detect functional errors, since these depend on the expected behavior of the program. If a tool fails to run or produces output that cannot be parsed, the failure is recorded as a tool error and no prediction is derived from it.

3.10 Interactive Cycles with Tools

Two experiments use the static tools in their loop: one for code generation and one for code verification. Figures 2 and 3 summarize both flows.

3.10.1 For Code Generation

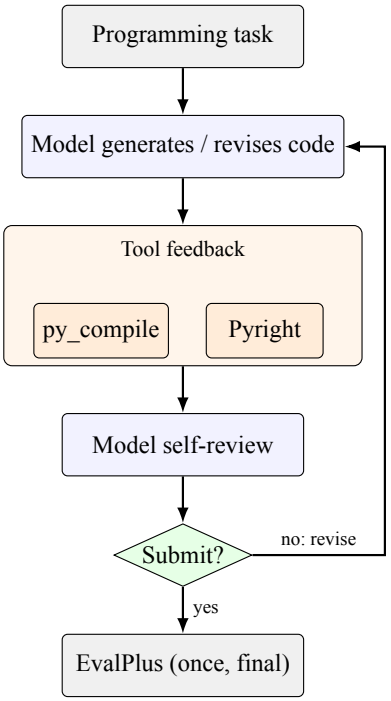


Figure 2. Code generation cycle. The model iterates between generation, static analysis, and self-review, and EvalPlus is applied only to the final candidate.

In the generation cycle, the model first produces an initial solution for a programming task. In each round, the candidate is analyzed by Python compilation and, only when it compiles, by Pyright, so the tools run in a short-circuit order and a candidate that does not compile does not reach Pyright. The resulting diagnostics are returned to the model, which performs a self-review and may either submit the current solution or return a revised implementation, following the broader paradigm of iterative self-feedback and self-debugging (Madaan et al., 2023; Chen et al., 2024). The prompt contains only the task, the current candidate, and the current feedback; the full history of previous rounds is not provided. When no diagnostic is found, the feedback states

that no syntax or statically detectable runtime issue was reported and still asks the model to assess the code, so the model may revise even without diagnostics. When the model revises, the returned implementation completely replaces the candidate, without any correctness validation before the substitution, and the cycle repeats, and when the model does not return the requested JSON, a code-only fallback is used. The cycle stops when the model submits or when the maximum of five rounds is reached, and both the generation and the review use a temperature of 0. The evaluation with EvalPlus is applied only once, to the final candidate, and its result is never included in any generation or self-review prompt, so the model has no access to the test outcome during the cycle. Because a revision is not guaranteed to be better than the previous candidate, the process is also analyzed in terms of improved, unchanged, and regressed outcomes, and regressions are retained rather than discarded.

3.10.2 For Code Verification

In the verification experiment, the model receives the programming task description and a held-out solution generated during the initial code generation, together with the static feedback produced by Python compilation and Pyright for that solution. This experiment is a controlled comparison: the same model, on the same sample, is evaluated with and without the static feedback. The version without feedback corresponds to the general error detection experiment, and the two versions share the same task, the same code, the same base prompt, the same temperature, and the same metrics, so only the static feedback differs. The feedback is presented as supporting evidence and never as a ground truth label. It is a short text that reports the compile status and, when the source compiles, the Pyright runtime status, followed by the diagnostics; when no issue is found, it states that no syntax or statically detectable runtime issue was reported and adds that this does not prove functional correctness. The prompt states that the tools can be incomplete, that the absence of a diagnostic does not imply that the code is correct, and that the tools do not detect functional errors in general. The model then returns the multilabel classification of the code into syntax, runtime, functional, and correct. The static feedback is used only as evidence for the classification and never replaces the reference labels, which are used exclusively afterward to compute the metrics.

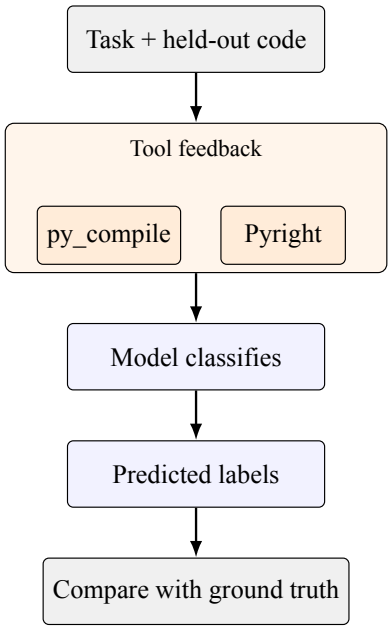


Figure 3. Code verification cycle. The model classifies held-out code using the static feedback as supporting evidence.

4 Results and Discussion

4.1 RQ1: To what extent can different language models generate correct code?

The first question concerns the ability of language models to generate valid code, with the large models included as external comparison baselines. The observed Correct proportions range from 62.2% to 80.8%. Muse Glimmer 30B has the highest point estimate (455/563; 80.8%, 95% CI [77.4, 83.9]), while Gemma 3 4B has the lowest (350/563; 62.2%, 95% CI [58.1, 66.1]). GPT-OSS 20B and Gemma 4 31B reach 79.0% (445 tasks) and 78.7% (443 tasks), respectively. These intervals quantify task-to-task uncertainty within EvalPlus, not sampling uncertainty over arbitrary programming work.

Regarding model size and performance, larger parameter counts do not consistently correspond to higher observed correctness among these models. A notable example is the Qwen 2.5 Coder family: the 7B model reaches 73.4% (413 tasks), compared with 67.1% (378 tasks) for the 32B model. The paired difference is 6.2 percentage points (95% bootstrap CI [2.3, 10.1]); the 7B model alone solves 82 tasks and the 32B model alone solves 47. The exact McNemar result is $p = 0.0026$ before correction and $p_{\mathrm{Holm}} = 0.091$ across all 78 pairs. Thus, the estimated effect is positive on this benchmark, but the post-hoc family does not support declaring this pair different at the 0.05 threshold; this is not evidence of equivalence.

The large-model baselines reinforce this weak relationship between parameter count and code-generation performance. DeepSeek V4.1 Flash and MiMo V2.5 each solved 413 of the 563 tasks (73.4%), exactly matching the number of correct solutions produced by Qwen 2.5 Coder 7B. Their correctness was also close to Qwen 3.8 27B (76%) and Devstral Small 2 24B (71%), while remaining below the best SLMs, Muse Glimmer 30B (81%), GPT-OSS 20B (79%), and Gemma 4 31B (79%). Thus, for these short function-generation tasks, the much larger total parameter counts of DeepSeek and MiMo did not produce a correspondingly large advantage. The overlap between the two groups indicates that SLMs and LLMs are not cleanly separated by correctness on this benchmark.

Output-contract compliance further qualifies this comparison. DeepSeek produced 33 Syntax outcomes (5.9%), all on HumanEval tasks and all involving indentation failures; in 32 of these cases, the response omitted a function definition and returned an indented function body instead. MiMo exhibited the same pattern in 10 cases (1.8%). By contrast, most SLMs kept Syntax outcomes between 0% and 3%, despite their lower parameter counts. This suggests that sev-

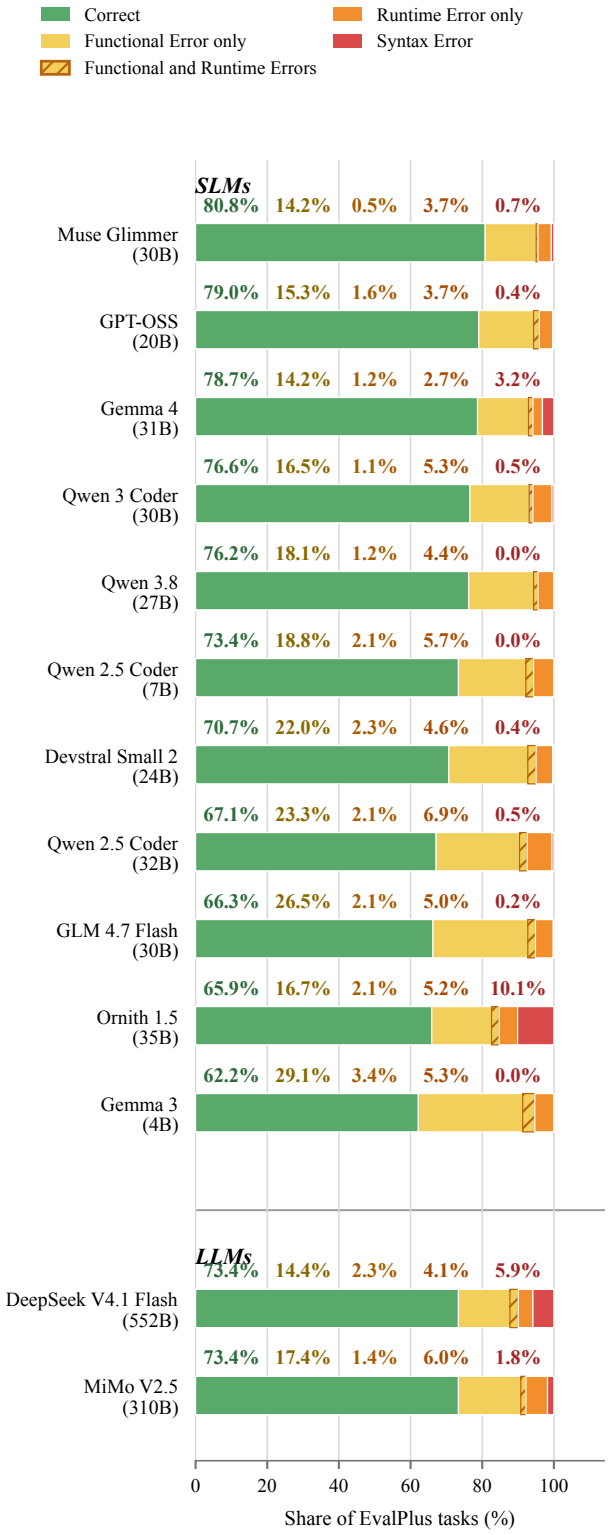


Figure 4. Code generation outcomes across the 563 EvalPlus tasks. Each horizontal bar partitions the tasks into mutually exclusive outcomes; diagonal hatching marks programs labeled as both Functional and Runtime. The five evenly spaced values above each bar report, respectively, Correct, Functional Error only, Functional and Runtime Errors, Runtime Error only, and Syntax Error.

eral SLMs understood the required output format more reliably than the substantially larger DeepSeek baseline. For this evaluation, robust instruction following and adherence to the benchmark interface were at least as important as model scale: a semantically plausible body is unusable when it does not satisfy the expected function-level response contract.

Functional Error is the predominant failure mode across all models, ranging from 15% to 33%. Runtime Error appears with much lower frequency, ranging from 4% to 9%. This pattern indicates that the main challenge is not producing Python code that cannot execute, but rather producing executable solutions that do not fully satisfy the task specification.

Syntax Errors are rare for most models (0–3%), with Ornith 1.5 35B being a clear outlier at 10%. However, inspection revealed that 91.2% of Ornith's cases classified as Syntax Error correspond to failures in the expected output structure, representing 9.2% of all its generations. The benchmark expects a complete function definition, but the model sometimes returns only the function body or another incompatible structure. According to the classification procedure adopted in this study, this belongs to the Syntax Error category, although the underlying mechanism is frequently an inconsistency in the output format rather than a syntactically malformed Python token. Gemma 4 31B also exhibited formatting issues, with 72.2% of its Syntax Errors associated with output formation problems, impacting 2.3% of all generations. Overall, only 20% of the errors classified as Syntax Error were actual syntax errors, corresponding to approximately 0.29% of all generations.

The exploratory Cochran Q comparison rejects equal marginal Correct probabilities across the 13 paired conditions ($Q = 230.97$, $df = 12$, $p < 0.001$). Exact McNemar post-hoc comparisons show, for example, that Muse Glimmer exceeds Gemma 3 by 18.7 points (95% CI [14.9, 22.4], $p_{\text{Holm}} < 0.001$), GLM by 14.6 points (95% CI [11.0, 18.3], $p_{\text{Holm}} < 0.001$), and Ornith by 14.9 points (95% CI [11.2, 18.5], $p_{\text{Holm}} < 0.001$). Its 1.8-point difference from GPT-OSS is uncertain (95% CI [-0.5, 4.3], $p_{\text{Holm}} = 1.000$); this result does not establish equivalence.

In synthesis, the evaluated models normally produce structurally executable Python code, and syntax errors are relatively rare, apart from model-specific output-contract failures. Functional failures are much more important than syntax failures, indicating that models frequently produce plausible and executable code that does not correctly implement the task specification. The two large-model baselines do not define a distinct upper range on these tasks: several compact-model point estimates overlap with or exceed them. Because scale is confounded with model family and backend, this is descriptive rather than a causal scale conclusion.

4.2 RQ2: To what extent can SLMs correctly identify errors in generated code?

Table 4 presents the overall error detection performance of the models. In the table, CR, FR, RR, and SR denote the recall for the Correct, Functional, Runtime, and Syntax classes, respectively. The columns marked with (NS) report the met-

Table 4. General error detection results by model (%). CR, FR, RR, and SR denote Correct, Functional, Runtime, and Syntax recall. NS indicates metrics computed without the Syntax class.

Model	Acc	MF1	MR	CR	FR	RR	SR	MF1(NS)	MR(NS)
General-purpose models									
Muse Glimmer 30B	73.1	53.8	54.7	83.3	55.5	27.0	52.9	61.3 (+7.5)	55.3 (+0.6)
Gemma 4 31B	72.4	48.5	59.1	83.7	50.1	24.1	78.4	55.8 (+7.3)	52.6 (-6.5)
Qwen 3.8 27B	62.6	42.6	54.0	62.9	75.9	12.7	64.7	48.6 (+5.9)	50.5 (-3.6)
Qwen 2.5 Coder 7B	62.5	33.7	34.8	72.3	46.7	2.5	17.6	39.4 (+5.7)	40.5 (+5.7)
Qwen 2.5 Coder 32B	60.8	41.0	49.3	71.1	38.1	11.4	76.5	43.3 (+2.3)	40.2 (-9.1)
Qwen 3 Coder 30B	60.6	37.8	40.1	63.8	65.9	9.3	21.6	44.1 (+6.3)	46.3 (+6.2)
Ornith 1.5 35B	56.3	35.7	44.2	55.4	78.1	8.0	35.3	43.1 (+7.4)	47.2 (+3.0)
Devstral Small 2 24B	53.9	38.1	42.9	54.9	63.6	17.7	35.3	43.3 (+5.2)	45.4 (+2.5)
GLM 4.7 Flash 30B	50.4	37.3	58.4	49.3	64.0	26.2	94.1	43.5 (+6.3)	46.5 (-11.9)
Gemma 4 E2B IT	48.1	33.1	33.9	47.8	65.4	16.5	5.9	40.6 (+7.5)	43.2 (+9.3)
Gemma 4 E4B IT	45.9	37.0	50.8	38.0	86.4	31.6	47.1	41.8 (+4.7)	52.0 (+1.2)
Gemma 3 4B IT	38.5	26.7	32.0	30.0	82.0	10.1	5.9	32.4 (+5.7)	40.7 (+8.7)
Gemma 3 4B	32.9	23.4	32.8	22.6	84.7	6.3	17.6	27.3 (+3.9)	37.9 (+5.1)
GPT-OSS 20B	25.1	39.0	44.7	17.5	55.8	32.9	72.5	40.4 (+1.4)	35.4 (-9.3)
Fine-tuned models									
Best Acc	77.8	50.9	48.5	85.3	69.1	27.8	11.8	63.1 (+12.2)	60.8 (+12.3)
Weighted MF1	73.5	54.2	62.9	75.9	77.9	32.9	64.7	62.9 (+8.7)	62.2 (-0.6)
Balanced Acc	67.8	38.1	38.9	74.4	65.8	15.2	0.0	50.9 (+12.7)	51.8 (+13.0)
Balanced MF1	63.7	36.0	37.1	68.6	65.8	13.9	0.0	48.0 (+12.0)	49.4 (+12.4)
Large models									
DeepSeek v4.1 Flash	75.5	51.0	47.7	89.3	51.6	14.8	35.3	54.1 (+3.1)	51.9 (+4.1)
MiMo v2.5	51.5	39.7	61.1	47.2	79.7	21.5	96.1	44.7 (+5.0)	49.5 (-11.7)

rics computed without the Syntax class. All values are percentages.

Muse Glimmer 30B has the highest aggregate point estimates among the general-purpose models, with 73.1% Accuracy (task-clustered 95% CI [68.0, 78.1]) and 53.8% Macro-F1 (95% CI [47.1, 60.5]). Gemma 4 31B follows with 72.4% Accuracy (95% CI [67.1, 77.5]) and 48.5% Macro-F1 (95% CI [43.5, 53.4]). RQ2 is descriptive and does not define a focal pairwise ranking test; overlapping intervals are therefore presented as uncertainty, not as an equivalence or significance decision. No single model dominates all classes, and different metrics produce different rankings.

The Qwen 2.5 Coder family illustrates this point. The 7B model has higher Accuracy than the 32B model (62.5% versus 60.8%), but the 32B model has higher Macro-F1 (41.0% versus 33.7%) because it is much more balanced across the minority classes, particularly Runtime Recall (11.4% versus 2.5%) and Syntax Recall (76.5% versus 17.6%). This reinforces that model size alone does not determine detection quality and that Accuracy alone is insufficient to characterize performance in this multilabel setting.

Detection behavior is strongly class-dependent. Functional Error is the class for which several models show the highest sensitivity. Gemma 4 E4B reached 86.4% Functional Recall, followed by Gemma 3 4B (84.7%), Gemma 3 4B IT (82.0%), Ornith 1.5 (78.1%), and Qwen 3.8 (75.9%). However, high sensitivity to Functional Errors does not imply good overall performance. Gemma 3 4B, for instance, reached 84.7% Functional Recall but only 32.9% Accuracy, 22.6% Correct Recall, and 6.3% Runtime Recall, indicating difficulty with the remaining classes.

Runtime Error is the most difficult class. The highest general-purpose point estimate is GPT-OSS at 32.9% (95% CI [21.2, 50.5]), followed by Gemma 4 E4B (31.6%), Muse Glimmer (27.0%), GLM 4.7 Flash (26.2%), and Gemma 4 31B (24.1%). The broad interval around GPT-OSS reflects the small effective subgroup and prevents a precise ranking, but no general-purpose point estimate exceeds approximately one third of the Runtime cases.

Syntax Error shows large variability across models. GLM 4.7 Flash reached 94.1% Syntax Recall, followed by Gemma 4 31B (78.4%), Qwen 2.5 Coder 32B (76.5%), and GPT-OSS (72.5%). In summary, Functional Errors are relatively easy for some models, Runtime Errors are consistently difficult, and Syntax Errors present a strongly model-dependent behavior.

Some model-specific behaviors are particularly notable. GPT-OSS was one of the best code generators in RQ1, with 79% correctness, but it obtained only 25.1% Accuracy, 17.5% Correct Recall, 32.9% Runtime Recall, and 72.5% Syntax Recall in detection. This indicates that GPT-OSS is a good generator but weak at recognizing correct code. Among the 2700 samples whose ground truth is Correct, GPT-OSS classified 2227 (82.5%) as incorrect. It is, however, relatively strong in Runtime and Syntax, and weak specifically in recognizing Correct.

The comparison between the quantized and full-precision variants of Gemma 3 4B is also informative. The quantized

version (Q4_K_M) obtained 32.9% Accuracy and 23.4% Macro-F1, while the full-precision version (FP16) obtained 38.5% Accuracy and 26.7% Macro-F1. The differences by class were 22.6% versus 30.0% for Correct Recall, 84.7% versus 82.0% for Functional Recall, 6.3% versus 10.1% for Runtime Recall, and 17.6% versus 5.9% for Syntax Recall. Quantization reduces performance, but without a very large degradation, although the drop in Accuracy and Correct Recall is noticeable.

The influence of the Syntax class on the metrics deserves explicit discussion. Removing Syntax barely changes Accuracy, as shown by Muse Glimmer (73.1% to 73.2%), Gemma 4 31B (72.4% to 72.6%), Qwen 3.8 (62.6% to 62.8%), and GPT-OSS (25.1% to 25.3%). Macro-F1, on the other hand, is affected much more clearly, as shown by Muse Glimmer (53.8% to 61.3%), Gemma 4 31B (48.5% to 55.8%), Qwen 3.8 (42.6% to 48.6%), and Qwen 2.5 Coder 7B (33.7% to 39.4%). Because Syntax is rare, it barely affects Accuracy, but since Macro-F1 weights all classes similarly, poor or unstable Syntax performance strongly affects this metric. This justifies why Accuracy should not be analyzed alone, why macro metrics are important in an imbalanced dataset, and how the NS metrics help quantify the influence of Syntax on the evaluation.

Table 5 reports the self-error detection results. This table does not measure general detection capability; it measures, among the errors produced by the model itself, how many it is able to recognize. The overall Self-Det. is computed as the number of self-errors identified divided by the total number of self-errors, and each error type column reports the same rate restricted to that category. The line marked CI gives the lower and upper bounds of the 95% confidence interval for Self-Det.; it expresses the uncertainty around the percentage and is not an additional score. ND indicates that the model produced no errors of that category, so the self-detection rate is undefined.

The overall self-error detection point estimates range from Qwen 3.8 at 67.9% (55/81; 95% Wilson CI [57.1, 77.1]) to GPT-OSS at 5.3% (3/57; 95% CI [1.8, 14.4]). Gemma 3 reaches 60.4% (87/144; 95% CI [52.3, 68.0]), while GLM, Qwen 2.5 Coder 7B, and Ornith reach between 53.8% and 55.0%. These are descriptive proportions over each model's own and differently sized error subset. GPT-OSS is the clearest contrast: 79.0% Correct generation in RQ1 but only 5.3% exact-set self-error detection. Strong generation therefore does not imply strong self-evaluation under this protocol.

Functional Errors tend to be easier in self-evaluation, with Ornith reaching 80.0%, Gemma 3 77.3%, and Qwen 3.8 75.4%. Runtime estimates remain low and unstable: Qwen 3.8 and Muse Glimmer each detect 3/12 (25.0%), GLM detects 3/15 (20.0%), and Qwen 2.5 Coder 32B detects 7/36 (19.4%), while several models detect none.

Qwen 3.8 deserves an explicit contrast. In general detection it obtained only 12.7% Runtime Recall, but in self-error detection it reached 25.0%. It is therefore not the best general Runtime Error detector, but it is the best at recognizing the Runtime Errors it produced itself. GLM 4.7 Flash recognized 100% of its own Syntax Errors, but this corresponds to only 3 of 3 cases, and the denominator should always be considered.

Table 5. Self-error detection by each generator model. Self-Det. is exact-set detection; CI shows its 95% Wilson confidence interval. Class cells are detected/reference counts. ND indicates no reference error of that category.

Model	Self-Det.	Functional	Runtime	Syntax
Qwen 3.8 27B	67.9% (55/81) CI: 57.1–77.1	75.4% (52/69)	25.0% (3/12)	ND
Gemma 3 4B	60.4% (87/144) CI: 52.3–68.0	77.3% (102/132)	0.0% (0/30)	ND
GLM 4.7 Flash 30B	55.0% (61/111) CI: 45.7–63.9	65.6% (63/96)	20.0% (3/15)	100.0% (3/3)
Qwen 2.5 Coder 7B	54.4% (49/90) CI: 44.2–64.3	66.7% (46/69)	14.3% (3/21)	ND
Ornith 1.5 35B	53.8% (63/117) CI: 44.8–62.6	80.0% (48/60)	16.7% (4/24)	38.5% (15/39)
Devstral Small 2 24B	52.3% (58/111) CI: 43.0–61.3	64.4% (58/90)	9.1% (3/33)	ND
Qwen 3 Coder 30B	42.3% (33/78) CI: 32.0–53.4	51.5% (34/66)	0.0% (0/15)	ND
Qwen 2.5 Coder 32B	33.3% (37/111) CI: 25.3–42.5	36.9% (31/84)	19.4% (7/36)	ND
Muse Glimmer 30B	19.0% (12/63) CI: 11.2–30.4	25.9% (14/54)	25.0% (3/12)	ND
Gemma 4 31B	13.6% (9/66) CI: 7.3–23.9	6.7% (3/45)	16.7% (3/18)	33.3% (3/9)
GPT-OSS 20B	5.3% (3/57) CI: 1.8–14.4	15.7% (8/51)	0.0% (0/21)	ND

Answer to RQ2. SLMs exhibit heterogeneous and strongly class-dependent error-detection capabilities. Functional Errors are generally easier to identify, whereas Runtime Errors remain challenging for all evaluated models. No single model dominates across all categories, and strong code-generation performance does not necessarily translate into strong error detection or self-evaluation capability.

4.3 RQ3: To what extent can specialized SLMs improve the detection of errors in generated code?

The dataset used for training is strongly imbalanced. Correct samples represent 72.4% of the records, followed by Functional Error with 21.3%, Runtime Error with 6.6%, and Syntax Error with only 1.5%. Because Accuracy favors predictions of the dominant class, a model trained on this distribution can learn to play safe and predict Correct most of the time. This motivated the evaluation of different training strategies. The split is performed at the task level, so all samples of a given programming task belong to a single split, and balancing is applied only to the training split, while validation and test remain intact.

Table 6 reports the paired primary comparisons against Gemma 4 E4B. The estimates use only common successful rows and therefore can differ by at most a tenth of a point from the standalone coverage estimates in Table 4. All effects are specialized minus base, with tasks as clusters. Holm correction covers the eight Accuracy/Macro-F1 comparisons shown.

The first search optimized validation Accuracy using the standard causal loss and the original class distribution. The

Table 6. Paired base-versus-specialized comparisons on 3,726 common rows from 113 held-out tasks. Effects and task-bootstrap 95% CIs are percentage points; p_{H} is the two-sided permutation p-value after Holm correction.

Specialized model	Metric	Base	Specialized	Effect [95% CI]	p_{H}
Best Acc	Accuracy	45.9	77.9	+32.0 [25.0, 39.2]	$< .001$
Best Acc	Macro-F1	37.1	51.0	+13.9 [6.7, 21.0]	$< .001$
Weighted MF1	Accuracy	45.9	73.6	+27.7 [21.3, 34.2]	$< .001$
Weighted MF1	Macro-F1	37.1	54.3	+17.2 [11.9, 22.3]	$< .001$
Balanced Acc	Accuracy	45.9	67.9	+22.0 [15.0, 29.1]	$< .001$
Balanced Acc	Macro-F1	37.1	38.1	+1.0 [-4.6, 6.6]	1.000
Balanced MF1	Accuracy	45.9	63.8	+17.9 [11.4, 24.5]	$< .001$
Balanced MF1	Macro-F1	37.1	36.0	-1.1 [-6.5, 4.4]	1.000

search explored the base model, LoRA rank, alpha, dropout, learning rate, epochs, and bias configuration. The best configuration used Gemma 4 E4B with $r = 16$, $\alpha = 16$, dropout 0.1, learning rate 1×10^{-4}, 3 epochs, and bias `none`, reaching a validation Accuracy of 66.6%. Figure 5 shows that the Accuracy-oriented model shifts its decision boundary toward the dominant Correct class. In the base model, only 38.3% of the Correct samples remain Correct, while 49.2% are confused with Functional. Best Acc reduces Correct to Functional from 49.2% to 13.1% and also reduces Correct to Runtime (8.1% to 0.6%) and Correct to Syntax (4.4% to 0.8%), which explains where the Accuracy gain comes from. The cost is visible in the reverse direction: Functional to Correct rises from 4.5% to 25.2%, Runtime to Correct from 15.8% to 34.2%, and Syntax to Correct from 6.2% to 37.5%. The model did not simply learn Correct better; it began to favor Correct as an output.

Because per-class performance remained unbalanced, a second search optimized Macro-F1 instead. This search kept the same data distribution without explicit balancing and used the `label_weighted` strategy with a class weight of 3.0. The best configuration used Gemma 4 E4B with $r = 8$, $\alpha = 8$, dropout 0.1, learning rate 1×10^{-4}, 3 epochs, and bias `none`, reaching a validation Macro-F1 of 52.4% and a validation Macro Recall of 54.9%.

The comparison between the two optimization objectives, presented in Table 4 and in Figure 5, clarifies how strongly the training objective changes the detector profile. Best Acc maximizes global performance, with 77.8% Accuracy, 50.9% Macro-F1, and 85.5% Correct Recall. Weighted MF1 reduces Correct Recall to 76.2% but increases Functional Recall from 73.2% to 81.0%, Runtime Recall from 27.8% to 32.9%, and Syntax Recall from 12.5% to 68.8%. The confusion matrices show that Weighted MF1 also reduces the tendency of Best Acc to absorb minority classes into Correct: Functional to Correct falls from 25.2% to 16.2%, and Syntax to Correct falls from 37.5% to 6.2%. In contrast to Best Acc, the Macro-F1-oriented model partially reverses the tendency to collapse minority classes into Correct. The optimization objective therefore changes not only the aggregate score but the classes the detector learns to recognize, and this is one of the main findings of this RQ.

Runtime remains the least stable class across training strategies. In the base model, Runtime is predicted as Functional in 48.7% of cases and as Runtime in 32.9%. Best Acc shifts Runtime toward Correct (34.2%) and keeps only 27.8% as Runtime, while Weighted MF1 keeps 32.9% as Runtime and sends 30.4% to Correct and 34.2% to Functional. Fine-tuning changes where Runtime samples are misclassified, but does not produce an improvement comparable to that observed for Correct or Syntax. Syntax, in turn, is highly sensitive to the training strategy: it ranges from 11.8% recall under Accuracy-oriented training to 64.7% under Macro-F1-oriented training.

A third strategy changed the training distribution itself, reducing the number of Correct examples, removing Syntax, and testing proportions such as 1:1:1 and 1:3:3, with the goal of increasing the relative representation of the minority classes. The Balanced Acc model reached 67.8% Accuracy, 38.1% Macro-F1, 74.4% Correct Recall, 65.8% Functional Recall, 15.2% Runtime Recall, and 0% Syntax Recall. The Balanced MF1 model reached 63.7% Accuracy, 36.0% Macro-F1, 68.6% Correct Recall, 65.8% Functional Recall, 13.9% Runtime Recall, and 0% Syntax Recall.

The balanced strategies have lower Accuracy and Macro-F1 point estimates than the two original-distribution strategies; Runtime detection also decreases, and Syntax reaches 0% because that class was removed from training. Against the base, their paired Macro-F1 CIs include zero (Table 6). Thus, the evaluated rebalancing does not show a supported Macro-F1 benefit, whereas objective/loss changes on the original distribution do. This result is specific to the sampled ratios, loss, and Gemma base and does not rule out other balancing designs.

Syntax deserves separate discussion. The class is extremely rare, and many cases classified as Syntax result from an incompatible output format or structure rather than syntactically invalid Python. This makes the class small, heterogeneous, and difficult to learn, and highly sensitive to the training objective. The results illustrate this sensitivity: Best Acc reached 11.8% Syntax Recall, Weighted MF1 reached 64.7%, and the balanced models reached 0%. These cases are not reclassified; the observed behavior only reflects how the training strategy interacts with a rare and heterogeneous class.

The paired analysis quantifies the overall specialization effect. Best Acc gains 32.0 Accuracy points (95% CI [25.0, 39.2], $p_{\mathrm{H}} < .001$) and 13.9 Macro-F1 points (95% CI [6.7, 21.0], $p_{\mathrm{H}} < .001$). Weighted MF1 gains 27.7 Accuracy points (95% CI [21.3, 34.2]) and 17.2 Macro-F1 points (95% CI [11.9, 22.3]), both with $p_{\mathrm{H}} < .001$. The balanced models retain positive Accuracy effects, but their Macro-F1

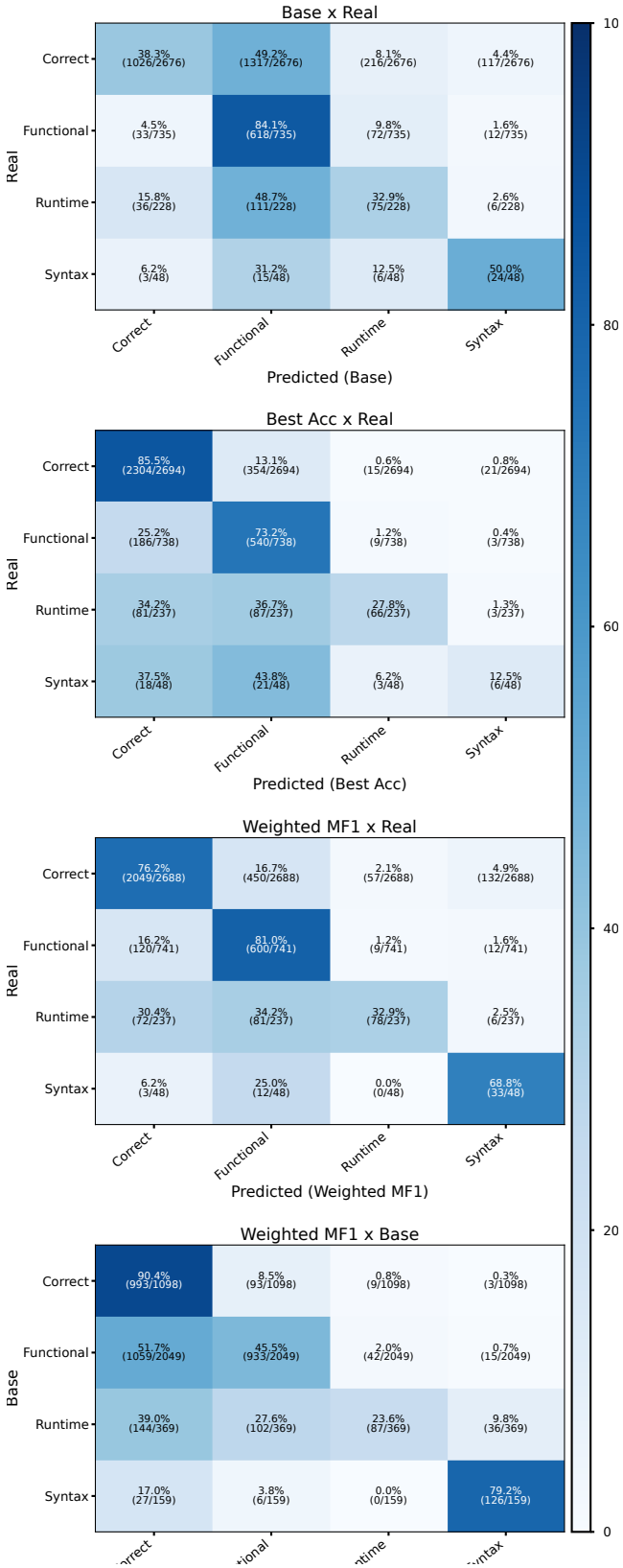


Figure 5. Row-normalized confusion matrices under a primary-label reduction (priority syntax, runtime, functional, correct). Each cell reports the proportion and the counts as predicted / reference total. The first three matrices use the ground truth as reference, and the last compares the Weighted MF1 predictions against the base predictions. The number of evaluated samples differs slightly across matrices (Base 3,687; Best Acc 3,717; Weighted MF1 3,714; Weighted MF1 vs Base 3,675) because unparsable model outputs were excluded from the primary-label reduction.

differences are uncertain and include zero. The class-level changes remain descriptive: Accuracy-oriented training increases Correct Recall while reducing Functional and Syntax recall, whereas Weighted MF1 yields a more balanced profile. Thus, the evidence supports specialization for the two original-distribution objectives but does not support a Macro-F1 gain from the evaluated balancing strategies.

The last matrix in Figure 5 compares the Weighted MF1 predictions directly against the base predictions and should be interpreted as a prediction transition, not as correctness. It shows that among the samples the base model labeled as Correct, 90.4% remain Correct; among those labeled as Functional, 51.7% are reclassified as Correct by Weighted MF1; among those labeled as Runtime, only 23.6% remain Runtime; and among those labeled as Syntax, 79.2% remain Syntax. The largest behavioral shift therefore occurs among samples originally labeled as Functional by the base model, which are frequently reclassified as Correct. This does not imply an error by itself; it only shows how the decision behavior changed. Fine-tuning does not simply improve the detector: it changes its decision behavior, and different optimization objectives produce distinct error profiles.

Answer to RQ3. On this held-out EvalPlus split, Accuracy- and Macro-F1-oriented specialization produce positive paired Accuracy and Macro-F1 effects with CIs excluding zero after multiplicity correction. The resulting detector is sensitive to the objective: Accuracy-oriented fine-tuning favors Correct, whereas Macro-F1-oriented training is more balanced across classes. The evaluated explicit balancing strategies improve Accuracy but do not show a supported Macro-F1 gain over the base model.

4.4 RQ4: To what extent can static analysis tools complement SLMs in code generation and error detection?

4.4.1 Static Analysis Tools as an Error-Detection Baseline

The first part of this RQ evaluates how traditional static-analysis tools perform independently from language models. Table 7 focuses on the two tools directly aligned with the taxonomy: Python compilation for Syntax and Pyright for statically detectable Runtime. Python compilation detects 17/17 Syntax cases in the held-out set (100.0%, 95% Wilson CI [81.6, 100.0]) and 90/90 globally (100.0%, 95% CI [95.9, 100.0]). The finite lower bounds emphasize that a perfect observed rate is not universal proof. General-purpose SLM Syntax point estimates in RQ2 range from 5.9% to 94.1%; within this benchmark, deterministic parsing provides a more consistent syntax signal.

Runtime errors present a different scenario. Pyright detects 7/79 held-out Runtime cases (8.9%, 95% CI [4.4, 17.2]) and 45/408 globally (11.0%, 95% CI [8.3, 14.4]). These point estimates are below GPT-OSS's 32.9% Runtime Recall without feedback, although the tools and language models use different detection mechanisms. Static analysis therefore provides strong evidence for Syntax but only partial evidence for Runtime under the selected Pyright rule mapping.

Table 7. Standalone static-tool recall with Wilson 95% CIs. Test contains 1,243 generated programs from 113 tasks; Global contains 6,193 programs.

Tool	Target	Test	Global
Python compile	Syntax	100.0 [81.6, 100.0]	100.0 [95.9, 100.0]
Pyright	Runtime	8.9 [4.4, 17.2]	11.0 [8.3, 14.4]

4.4.2 Static-Analysis-Assisted Error Detection

The most consistent effect of providing static-analysis feedback to the detectors is observed in the Syntax class. Almost all models reach perfect or near-perfect Syntax Recall after receiving the tool output. Gemma 3 4B increases from 17.6% to 100.0%, Qwen 2.5 Coder 7B from 17.6% to 100.0%, Qwen 3 Coder 30B from 21.6% to 100.0%, Muse Glimmer 30B from 52.9% to 100.0%, GPT-OSS 20B from 72.5% to 100.0%, and Gemma 4 31B from 78.4% to 100.0%. This indicates that SLMs are generally able to incorporate explicit deterministic evidence when the tool output is directly aligned with the target class. The effect is particularly important for models that originally had very weak Syntax detection, since the external signal largely eliminates the need for the model to infer syntactic validity from the source code alone.

The specialized models reveal that explicit evidence is not always sufficient to recover a class that was weakly represented during training. The Balanced Accuracy model, which was trained without Syntax examples, remains at 0.0% Syntax Recall even with tool feedback. The Balanced MF1 model, also trained without Syntax, improves from 0.0% to 81.2%, but still remains below the near-perfect performance observed for the specialized models that retained Syntax during training. In contrast, Best Acc and Weighted MF1 both reach 100.0% Syntax Recall with static feedback. These contrasting results suggest that the behavior learned during specialization influences how effectively a model incorporates tool evidence, even when the evidence explicitly indicates the presence of the missing class.

Runtime shows the opposite pattern. Although Pyright provides some static Runtime evidence, it fires on only a small fraction of the Runtime failures (8.9% on the test set), and providing its diagnostics to the language models does not consistently improve Runtime Recall. In several cases, Runtime detection decreases: GLM 4.7 Flash drops by 17.3 percentage points, Weighted MF1 by 12.7, Gemma 4 E4B by 11.4, Best Acc by 8.9, GPT-OSS by 6.3, Devstral Small 2 by 6.3, Gemma 4 E2B by 6.3, and Qwen 3 Coder by 5.5. Only a few models show small improvements, such as Gemma 3 4B (+2.5) and Qwen 2.5 Coder 7B (+5.1). Therefore, the availability of useful static evidence does not imply that the model will integrate that evidence correctly into its classification. The contrast with Syntax is especially clear: syntactic evidence is both reliable and effectively incorporated, while runtime evidence is informative but substantially harder for the models to use consistently.

Static feedback also changes the behavior of the models on classes that are not directly determined by the tools. Several models increase Functional Recall while simultaneously decreasing Correct Recall. GLM 4.7 Flash loses 12.6 percentage points of Correct Recall while gaining 23.9 points of Functional Recall; Qwen 3 Coder loses 18.1 points of Correct Recall while gaining 19.7 points of Functional Recall; and Qwen 2.5 Coder 32B loses 27.1 points of Correct Recall while gaining 8.6 points of Functional Recall. The most pronounced example is Qwen 2.5 Coder 7B, whose Correct Recall decreases by 48.8 points while Functional Recall increases by 42.6 points, alongside improvements of 5.1 points in Runtime Recall and 82.4 points in Syntax Recall. Its substantial decrease in Exact-set Accuracy therefore does not reflect a uniform degradation across classes, but rather a strong redistribution of predictions between Correct and Functional. Although the static tools do not directly evaluate functional correctness, the models appear to use the presence or absence of diagnostics as broader evidence about whether a program should be considered correct.

Some models incorporate the additional evidence with a substantially more balanced profile. Muse Glimmer reaches 79.0% Accuracy, 71.2% Macro-F1, and 68.1% Macro Recall with tools, while maintaining 89.1% Correct Recall, 58.1% Functional Recall, 25.3% Runtime Recall, and 100.0% Syntax Recall. Gemma 4 31B reaches 76.7% Accuracy, 66.2% Macro-F1, and 66.5% Macro Recall, with 87.1% Correct Recall, 56.2% Functional Recall, 22.8% Runtime Recall, and 100.0% Syntax Recall. Among the specialized models, Weighted MF1 achieves 75.6% Accuracy, 68.3% Macro-F1, and 68.8% Macro Recall, while retaining relatively high Correct and Functional Recall and reaching 100.0% Syntax Recall. These cases show that tool feedback does not necessarily produce the strong Correct–Functional redistribution observed in other models, and that the ability to exploit the added evidence varies substantially across model families and training strategies.

The aggregate metrics reflect these class-specific changes rather than a uniform improvement in error reasoning. Macro-F1 increases considerably for several models, including Gemma 3 4B (+23.6), Balanced MF1 (+19.5), Gemma 4 31B (+17.7), Muse Glimmer (+17.4), and Best Acc (+17.3). However, these gains are strongly influenced by the large increase in Syntax Recall, while Runtime Recall frequently remains unchanged or decreases and Correct/Functional predictions may shift substantially. Exact-set Accuracy is correspondingly more heterogeneous, since a prediction is counted as correct only when the complete set of labels matches the ground truth. The aggregate improvements should therefore not be interpreted as uniform improvements across all error categories. Table 8 summarizes the variation of every detection metric relative to RQ2.

The paired data for the two hosted models permit an uncertainty analysis over 1,243 programs clustered in 113 tasks. For DeepSeek, feedback changes Accuracy by only +0.9 points (95% CI [-0.6, 2.3], $p_{\mathrm{Holm}} = 1.000$) but changes

Macro-F1 by +13.4 points (95% CI [8.0, 19.2], $p_{\text{Holm}} = .010$), largely alongside a +64.7-point Syntax Recall effect (95% CI [42.1, 86.7], $p_{\text{Holm}} = .013$). For MiMo, Macro-F1 changes by +8.0 points (95% CI [3.3, 12.7], $p_{\text{Holm}} = .016$), while Accuracy changes by -4.6 points (95% CI [-8.0, -1.3], $p_{\text{Holm}} = .078$). These results illustrate why class-balanced metrics and multiplicity-adjusted inference must accompany the rounded deltas. The SLM rows remain descriptive because paired observations were not available for this analysis; they are not assigned retrospective p-values.

4.4.3 Static-Analysis-Assisted Iterative Code Generation

The effect of tool feedback during iterative code generation is again strongest for Syntax, but the improvement is less uniform than in the classification setting. Several models reduce or eliminate syntactic failures: Gemma 4 31B decreases Syntax errors by 3.2 percentage points to 0.0%, Ornith 1.5 reduces them by 5.5 points, Qwen 2.5 Coder 32B by 0.4 points, Qwen 3 Coder by 0.4 points, and Devstral Small 2 by 0.2 points. Ornith is particularly relevant because it was the model with the highest Syntax error rate in the original generation experiment; the iterative process reduces this rate substantially, from approximately 10.1% to 4.6%, but does not eliminate the problem. Moreover, some models regress: GLM 4.7 Flash increases Syntax errors by 8.0 percentage points, GPT-OSS by 1.4, and Muse Glimmer by 1.6. The GLM result is especially notable because it was the strongest general-purpose Syntax detector without tools, reaching 94.1% Syntax Recall in RQ2, yet the iterative generation process increases its Syntax error rate. This contrast shows that recognizing a syntactic error and successfully repairing or avoiding it during generation are distinct capabilities.

Runtime behaves differently from the tool-assisted classification experiment. While Runtime Recall often decreased when models used Pyright feedback to classify code, the iterative generation cycle reduces the incidence of Runtime errors for most models. Qwen 2.5 Coder 32B reduces Runtime failures by 2.5 percentage points, Ornith 1.5 and Qwen 3.8 by 1.4 points, GPT-OSS by 1.1, Devstral Small 2 by 0.9, and Qwen 3 Coder by 0.7. Some models show no improvement or a small regression, such as Qwen 2.5 Coder 7B, which remains unchanged, and Gemma 4 31B, which increases Runtime errors by 0.4 points. This difference between classification and generation suggests that models may be more effective at using Pyright diagnostics as actionable repair feedback than at translating the same evidence into an explicit Runtime classification.

Changes in the proportion of Correct solutions should be interpreted together with the corresponding Functional, Runtime, and Syntax changes. Qwen 2.5 Coder 32B presents the clearest positive case: Correct increases by 8.2 percentage points while Functional decreases by 5.0, Runtime by 2.5, and Syntax by 0.4, indicating that the gain in correctness is distributed across multiple failure types. Ornith 1.5 increases Correct by 6.2 points while reducing Runtime by 1.4 and Syntax by 5.5, suggesting that its gain is primarily associated with eliminating structural and execution failures. Gemma 4 31B increases Correct by 3.2 points while Syntax decreases by exactly 3.2 points, Functional remains unchanged, and Runtime slightly increases, indicating that its improvement is largely associated with the elimination of syntactic failures.

The iterative process can also produce regressions. Qwen 2.5 Coder 7B loses 2.8 percentage points of Correct while Functional increases by 3.0, with no change in Runtime or Syntax. Qwen 3 Coder loses 2.3 points of Correct while Functional increases by 3.9, despite small reductions in Runtime and Syntax. Muse Glimmer loses 2.0 points of Correct, with a 0.9-point increase in Functional and a 1.6-point increase in Syntax. These cases show that improvements in statically observable properties do not necessarily translate into improved functional correctness, and that a revision guided by static feedback may replace a previously correct or partially correct solution with a worse candidate.

The average number of iterations also does not show a simple relationship with the final improvement. Qwen 2.5 Coder 32B requires only 1.36 cycles on average while achieving the largest gain in Correct (+8.2), whereas GLM 4.7 Flash averages 2.14 cycles and still loses 2.8 points of Correct while increasing Syntax errors. Qwen 2.5 Coder 7B similarly averages 2.06 cycles and loses 2.8 points of Correct. Additional revision opportunities therefore do not guarantee better final solutions; the outcome depends on how effectively the model interprets and acts on the tool feedback. Table 9 summarizes the variation of every generation metric relative to RQ1, together with the average number of cycles.

For the two hosted models with complete paired Phase-1/Phase-6 data, the Correct increase is supported after correcting the two-model family. DeepSeek improves by 7.3 points (95% CI [4.6, 10.1], exact McNemar $p_{\text{Holm}} < .001$; 13 Phase-1-only versus 54 Phase-6-only successes), and MiMo improves by 6.6 points (95% CI [3.9, 9.2], $p_{\text{Holm}} < .001$; 11 versus 48). Both have a median of one cycle and IQR [1, 1], despite means of 1.14 and 1.15. The SLM rows in Table 9 are retained as descriptive point estimates because paired observations were not available for this analysis.

Answer to RQ4. Static analysis complements SLMs unevenly across failure types. Syntactic hallucinations benefit most consistently: deterministic compilation feedback raises Syntax Recall to nearly 100% for most detectors and generally reduces syntax failures during iterative generation. Runtime errors remain substantially more difficult. Although Pyright provides a partial Runtime signal, its diagnostics do not consistently improve Runtime classification, even though they more frequently help reduce Runtime failures during iterative repair. Static feedback also affects the distinction between Correct and Functional predictions, sometimes substantially changing the decision behavior of the model despite neither tool directly evaluating functional correctness. Overall, tool assistance can improve both detection and generation, but its effectiveness is strongly model-dependent, and iterative revision may also introduce regressions.

Table 8. Detection results with static-analysis feedback, compared with the no-feedback setting of RQ2. Values are percentages and the value in parentheses is the change relative to RQ2.

Model	Acc	MF1	MR	CR	FR	RR	SR
General-purpose models							
Muse Glimmer 30B	79 (+6)	71 (+17)	68 (+13)	89 (+6)	58 (+3)	25 (-2)	100 (+47)
Gemma 4 31B	77 (+4)	66 (+18)	67 (+7)	87 (+3)	56 (+6)	23 (-1)	100 (+22)
Qwen 3.8 27B	69 (+7)	56 (+13)	63 (+9)	73 (+10)	69 (-6)	9 (-4)	100 (+35)
Qwen 2.5 Coder 7B	37 (-26)	47 (+13)	55 (+20)	24 (-49)	89 (+43)	8 (+5)	100 (+82)
Qwen 2.5 Coder 32B	43 (-18)	50 (+9)	50 (+1)	44 (-27)	47 (+9)	9 (-3)	100 (+24)
Qwen 3 Coder 30B	52 (-8)	46 (+9)	59 (+19)	46 (-18)	86 (+20)	4 (-5)	100 (+78)
Ornith 1.5 35B	59 (+2)	52 (+16)	62 (+17)	55 (-1)	84 (+6)	8 (+0)	100 (+62)
Devstral Small 2 24B	56 (+2)	53 (+15)	59 (+16)	54 (-1)	71 (+7)	11 (-6)	100 (+65)
GLM 4.7 Flash 30B	46 (-4)	45 (+7)	58 (+0)	37 (-13)	88 (+24)	9 (-17)	100 (+6)
Gemma 4 E2B IT	50 (+2)	49 (+16)	55 (+21)	48 (+0)	68 (+3)	10 (-6)	94 (+88)
Gemma 4 E4B IT	46 (+0)	52 (+15)	62 (+11)	35 (-3)	93 (+6)	20 (-11)	100 (+50)
Gemma 3 4B IT	36 (-2)	46 (+19)	52 (+20)	25 (-5)	83 (+1)	8 (-3)	94 (+88)
Gemma 3 4B	36 (+3)	47 (+24)	54 (+21)	24 (+2)	82 (-3)	9 (+3)	100 (+82)
GPT-OSS 20B	31 (+6)	50 (+11)	51 (+6)	26 (+8)	52 (-4)	27 (-6)	100 (+27)
Fine-tuned models							
Best Acc	77 (+0)	68 (+17)	67 (+19)	85 (+0)	65 (-4)	19 (-9)	100 (+88)
Weighted MF1	76 (+2)	68 (+14)	69 (+5)	79 (+4)	75 (-3)	20 (-13)	100 (+31)
Balanced Acc	67 (-1)	36 (-2)	38 (-1)	71 (-3)	71 (+5)	10 (-5)	0 (+0)
Balanced MF1	61 (-3)	56 (+20)	56 (+19)	62 (-7)	74 (+8)	7 (-7)	81 (+81)
Large models							
DeepSeek V4.1 Flash	76 (+1)	64 (+13)	63 (+16)	88 (-2)	55 (+4)	10 (-5)	100 (+65)
MiMo V2.5	47 (-4)	47 (+8)	59 (-2)	37 (-10)	90 (+10)	9 (-13)	100 (+4)

4.5 RQ5: How do SLMs compare with substantially larger LLMs across code generation, hallucination detection, and tool-assisted settings?

This RQ synthesizes whether the observed SLM results remain in the range of two substantially larger language models across code generation, hallucination detection, and static-analysis feedback. It is not a causal scale experiment: the two large models were purposively selected, and parameter count is confounded with architecture, training data, backend, provider revision, and numerical precision. We therefore reuse the effects and CIs from RQ1–RQ4 and do not add an omnibus "SLM versus LLM" test.

4.5.1 Code Generation

The code-generation point estimates place several SLMs in the same range as, or above, the two large-model baselines. Muse Glimmer reaches 80.8% Correct (95% CI [77.4, 83.9]), GPT-OSS 79.0% [75.5, 82.2], and Gemma 4 31B 78.7% [75.1, 81.9]. DeepSeek and MiMo each reach 73.4% [69.6, 76.8], exactly the Qwen 2.5 Coder 7B point estimate. These benchmark-conditioned estimates do not support a uniform ordering by scale; they do not establish that SLMs are generally superior outside these tasks.

The comparison also shows that larger scale does not eliminate structural or output-contract failures. Most SLMs produced Syntax errors in at most 3% of the generations, while DeepSeek exhibited a higher rate of approximately 6%, mostly caused by responses that did not follow the expected function-level output format. Thus, some SLMs not only match or exceed the correctness of the larger models, but also adhere more consistently to the required generation interface. In this setting, parameter count alone does not determine code-generation quality, and instruction following and output-contract compliance remain important even at much larger model scales.

These observations reinforce RQ1: within this single-generation EvalPlus run, a parameter increase of roughly an order of magnitude does not guarantee a larger Correct proportion. Repeated generations and a systematically sampled model set would be required to characterize a stable scale relationship.

4.5.2 Hallucination Detection

Hallucination-detection estimates overlap across scale groups. Muse Glimmer reaches 73.1% Accuracy (95% CI [68.0, 78.1]) and 53.8% Macro-F1 [47.1, 60.5], while the paired RQ3 analysis estimates Best Acc at 77.9% Accuracy and Weighted MF1 at 54.3% Macro-F1. DeepSeek reaches 75.5% Accuracy [70.1, 80.7] and 51.0% Macro-F1 [43.6, 58.0]. MiMo is lower in Accuracy but has a class profile distinct from the other models. These results show no clean separation between the selected SLM and large-model conditions; they are not a general population comparison of model sizes.

Table 9. Code generation with iterative static-analysis feedback, compared with RQ1. Values are percentages and the change relative to RQ1 is shown below each value. Avg Cyc is the mean number of cycles.

Model	Avg Cyc	Correct	Funct.	Runtime	Syntax
Small language models					
Gemma 4 (31B)	1.29	82 +3	15 0	4 0	0 -3
GPT-OSS (20B)	1.46	80 +1	15 -2	4 -1	2 +1
Muse Glimmer (30B)	1.41	79 -2	16 +1	4 0	2 +2
Qwen 3.8 (27B)	1.17	78 +2	19 -1	4 -1	0 0
Qwen 2.5 Coder (32B)	1.36	75 +8	20 -5	7 -2	0 0
Qwen 3 Coder (30B)	1.57	74 -2	21 +4	6 -1	0 0
Devstral Small 2 (24B)	1.37	73 +2	22 -2	6 -1	0 0
Ornith 1.5 (35B)	1.68	72 +6	19 +1	6 -1	5 -6
Qwen 2.5 Coder (7B)	2.06	71 -3	24 +3	8 0	0 0
GLM 4.7 Flash (30B)	2.14	63 -3	25 -4	5 -2	8 +8
Gemma 3 (4B)	1.43	61 -2	34 +1	8 -1	0 0
Large language models					
DeepSeek V4.1 Flash (552B)	1.14	81 +7	16 -1	5 -1	0 -6
MiMo V2.5 (310B)	1.15	80 +7	16 -2	5 -2	0 -2

The class-level results further show that model scale does not remove the main difficulties identified in RQ2. Runtime remains challenging for both groups: even the large models remain below the strongest specialized and general-purpose SLMs in some Runtime comparisons, and none of the evaluated models consistently achieves high Runtime Recall. By contrast, some SLMs remain particularly strong in Functional or Syntax detection. Therefore, the class-dependent behavior observed among SLMs persists when substantially larger models are considered, indicating that the difficulty of identifying a hallucination depends more on the failure type than on scale alone.

Specialization is particularly relevant in this comparison. Against their own base on matched data, Best Acc gains 32.0 Accuracy points (95% CI [25.0, 39.2]) and Weighted MF1 gains 17.2 Macro-F1 points [11.9, 22.3], with both adjusted $p < .001$. Their resulting point estimates fall within or above the range of the two general-purpose large models. This supports specialization in the evaluated Gemma setting; it does not isolate specialization from all other differences between compact and hosted models.

This comparison also strengthens one of the main findings of RQ2 and RQ3: generation capability and evaluation capability are distinct. Some SLMs that are strong generators remain weak detectors, while specialized SLMs improve substantially without any increase in parameter count. The comparison with larger models therefore provides no evidence that simply increasing scale resolves this gap. Instead, the results suggest that the training objective and task specialization play a central role in hallucination detection.

4.5.3 Tool-Assisted Error Detection

The use of static-analysis feedback does not create a clear scale advantage either. As observed for the SLMs in RQ4, the most consistent improvement occurs in Syntax detection. With tool feedback, most SLMs reach 100% or near-100% Syntax Recall, matching the large-model baselines. This indicates that once deterministic syntax evidence is explicitly available, compact models are generally able to exploit it as effectively as substantially larger models.

Runtime remains difficult even with external evidence. The addition of Pyright diagnostics does not consistently increase Runtime Recall for either group, and several SLMs experience decreases similar to those observed for the large-model baselines. This suggests that the limitation identified in RQ4 is not exclusive to compact models: even substantially larger models do not reliably translate partial static Runtime evidence into a correct Runtime classification. Scale therefore does not appear to solve the problem of integrating incomplete static evidence into an explicit error judgment.

The effect on Correct and Functional predictions is also similar across model scales. Some SLMs exhibit a strong redistribution between these classes after receiving static feedback, while the large-model baselines show comparable shifts. This reinforces the interpretation that models may use the presence or absence of static diagnostics as broader evidence about program correctness, even though the tools do not directly evaluate functional behavior. Importantly, the SLMs do not appear systematically more susceptible to this

effect than the much larger models.

Overall, the tool-assisted detection results show that SLMs remain competitive once external evidence is introduced. Their main limitations are largely shared by the larger models: Syntax evidence is easy to exploit, Runtime evidence is much harder to integrate, and static feedback can indirectly alter Functional and Correct predictions. Therefore, the effectiveness of tool-assisted hallucination detection appears to depend more on the type of evidence and the model's learned decision behavior than on parameter scale alone.

4.5.4 Tool-Assisted Iterative Code Generation

The clearest descriptive difference between groups appears in iterative code generation. Retained SLM point estimates are heterogeneous: Qwen 2.5 Coder 32B is about 8 Correct points above its Phase-1 value, Ornith about 6, Gemma 4 31B about 3, and Qwen 3.8 about 2, while other SLMs are unchanged or lower. These SLM deltas should be treated as descriptive and not as inferentially supported improvements.

The two large-model baselines both show supported gains in this setting: DeepSeek +7.3 points (95% CI [4.6, 10.1]) and MiMo +6.6 [3.9, 9.2], each with Holm-adjusted $p < .001$, while requiring little more than one cycle on average. The retained SLM point estimates are more heterogeneous and place Gemma 4 31B near 82% Correct and GPT-OSS/Muse near 80%. Since formal paired inference is available only for the two hosted models, the apparent difference in consistency between groups remains descriptive rather than a formal between-group result.

The error-type changes also reveal similarities between the groups. Both SLMs and larger models generally reduce Runtime and Syntax failures when tool feedback is used during generation, while Functional behavior remains more dependent on the individual model. Several SLMs obtain correctness gains by reducing a combination of Functional, Runtime, and Syntax failures, demonstrating that compact models can use tool feedback not only to repair structural problems but also to improve the final behavior of the generated program.

At the same time, the SLM results show greater variability and include clear regressions. Some models improve the statically observable properties of the code while increasing Functional errors or decreasing overall correctness. This suggests that larger models may be somewhat more stable when acting on iterative feedback, even though the final correctness of the best SLMs remains comparable. The distinction is therefore not that SLMs are unable to use tools, but that their response to iterative feedback is more model-dependent.

Across the evaluated settings, point estimates and CIs do not reveal a clean separation by scale. Several SLM conditions fall within or above the range of the two large baselines in generation and detection, and specialized Gemma detectors show supported improvements over their own base. The two large models both improve under iterative feedback, whereas the retained SLM point estimates vary in direction. This pattern motivates the hypothesis that scale may aid consistent use of feedback, but the current non-random two-model comparison cannot test that mechanism.

Answer to RQ5. On these EvalPlus tasks and under the recorded protocols, selected SLMs have generation and detection estimates in the same range as the two substantially larger baselines, and specialization produces supported within-model gains. Runtime limitations persist at both scales. Both large models improve during iterative tool-assisted generation, while SLM point estimates are more heterogeneous, but missing local paired raw data prevents a formal group comparison. The evidence is therefore benchmark- and model-conditioned and does not establish a general causal effect of scale.

5 Threats to Validity

This study has several limitations that should be considered when interpreting the results. We organize them according to construct, internal, external, conclusion, and reliability threats, followed by ethical considerations.

5.1 Construct Validity

The experiments use a fixed prompting configuration for code generation, hallucination detection, and tool-assisted interaction. Prompt engineering was not treated as an independent variable in this study. Consequently, the reported results characterize model behavior under the evaluated prompts and should not be interpreted as the maximum achievable performance of each model. Alternative system prompts, few-shot examples, explicit reasoning strategies, or model-specific prompt optimization could affect both generation and detection performance. We intentionally kept the prompts fixed because the objective was to compare models, specialization strategies, and tool assistance under controlled conditions rather than to optimize each model individually. Using model-specific prompting would introduce an additional source of variability and would make it harder to attribute differences to the factors investigated in the research questions.

The hallucination taxonomy adopted in this work is outcome-oriented and focuses on Syntax, Runtime, and Functional failures, in addition to Correct. This representation is useful because the classes can be grounded in observable execution outcomes, but it does not capture all possible dimensions of code hallucination. For example, the study does not explicitly model API hallucinations, dependency hallucinations, security issues, performance problems, maintainability, or code-quality defects. Furthermore, some classes are internally heterogeneous. In particular, Syntax includes both malformed Python code and responses that violate the expected output contract, such as returning an indented function body without the required function definition. The objective of the taxonomy is therefore not to describe the root cause of every failure, but to provide an operational classification suitable for automated execution-based evaluation.

The multilabel formulation also introduces an asymmetry between classes. Runtime and Functional may co-occur because different test cases can expose different failure modes, whereas Syntax terminates execution and Correct is mutually exclusive with all error labels. As a consequence, a syntactically invalid solution cannot be further evaluated for poten-

tial runtime or functional failures. This behavior follows the execution semantics of the evaluation pipeline and should be considered when comparing the prevalence of the different classes.

The confusion matrices reduce each multilabel set to one primary class solely for visualization, using the declared priority Syntax, Runtime, Functional, Correct. Their row-normalized percentages and per-row denominators are shown in the caption; this reduction does not replace the multilabel metrics used for evaluation.

Finally, the Correct label is determined by the available EvalPlus test suites. Although EvalPlus substantially strengthens the original HumanEval and MBPP test sets, passing all available tests does not constitute a formal proof of semantic correctness. A generated program may still fail on an input not covered by the benchmark. Therefore, Correct should be interpreted as correctness with respect to the available evaluation suite rather than universal correctness.

5.2 Internal Validity

The generation pipeline includes response normalization and code extraction before compilation and execution. Some responses may contain semantically meaningful code but fail to satisfy the expected output format, causing the harness to produce a Syntax outcome. This creates a potential instrumentation threat because the observed result depends on the interaction between the model output and the extraction procedure. However, adherence to the requested output contract is itself part of the evaluated task. The prompt requires a specific function-level response, and models that frequently return incompatible structures cannot be treated as equivalent to models that consistently produce executable outputs. Moreover, these failures are concentrated in specific models rather than uniformly distributed across the evaluated systems, suggesting that the phenomenon is not solely an artifact of the parser.

The evaluated models are also executed under different numerical configurations. Local models used for the initial generation experiments were generally quantized to make the large experimental matrix computationally feasible, while some model variants and hosted large models were executed under different precision settings. A limited comparison between quantized and non-quantized Gemma variants indicates that quantization can affect detection performance, but its effect is not systematically isolated for every model family. Consequently, some cross-model differences may partially reflect inference configuration in addition to model architecture and training.

The specialized detectors are based on Gemma-family models and use a particular QLoRA training pipeline. The observed improvements therefore demonstrate that specialization can be effective in the evaluated setting, but they do not establish that the same gains would necessarily be obtained with every SLM family or adaptation strategy. Similarly, the results regarding Accuracy-oriented, Macro-F1-oriented, and balanced training reflect the specific loss design, hyperparameter search space, and dataset construction used in this study.

The knowledge-distillation process introduces another potential source of noise. The ground-truth labels are always derived from execution, but the natural-language rationales used as auxiliary supervision are generated by three language models. These explanations may contain incomplete or inaccurate reasoning even when the associated label is correct. This threat is reduced by the design of the pipeline: teacher models do not determine or modify the reference labels, and only the predicted classes are used in the quantitative evaluation. Nevertheless, the textual quality of the rationales is not independently validated and may influence the fine-tuning process.

5.3 External Validity

The experiments are restricted to Python and to the HumanEval and MBPP tasks included in EvalPlus. These benchmarks primarily consist of relatively short function-level programming problems. The observed behavior may differ in other programming languages or software-engineering scenarios involving multiple files, external dependencies, build systems, persistent state, concurrency, frameworks, or repository-level context. In particular, failures involving APIs, dependency resolution, project configuration, and interactions across files may produce hallucination patterns not represented by the current taxonomy or benchmark.

The evaluated model set covers multiple architectures, sizes, and code-oriented models, but it remains a sample of the rapidly changing model landscape. The conclusions should therefore not be interpreted as statements about all SLMs or all large language models. Only two substantially larger models are used as external baselines, and the study intentionally focuses on SLMs rather than attempting an exhaustive scale study. The comparison is sufficient to examine whether a large parameter gap necessarily implies better performance under the evaluated protocol, but broader claims about scaling would require a larger and more systematically sampled set of models.

The static-analysis conclusions are similarly limited to the tools and diagnostics considered in the study. Python compilation provides a strong signal for Syntax, whereas Pyright covers only a subset of possible Runtime failures. The weak Runtime Recall of Pyright in this setting indicates that the results should not be generalized to static analysis as a whole. More sophisticated techniques, including symbolic execution, abstract interpretation, interprocedural analysis, or dynamic analysis, could provide different forms of evidence and might lead to different interactions with language models.

5.4 Conclusion Validity

Phase 1 uses one generation per model/task at temperature 1.0, provider-default top-p/top-k, and no fixed generation seed. The estimates therefore describe one stochastic draw and do not measure within-model sampling variance or pass@k. We mitigate overinterpretation by reporting task-level CIs and by conditioning claims on this run, but repeated seeded generations are needed to assess stability.

The 563 tasks constitute the full selected benchmark, not 563 independent draws from all software. Pairing controls task difficulty within direct comparisons, and task-cluster bootstrap/permutation preserves dependent rows from the same task, but independence between task clusters remains an approximation. Multiple comparisons can inflate false positives; the analysis therefore defines Holm families for all 78 RQ1 pairs, eight RQ3 primary comparisons, and the available RQ4 comparisons. All tests are two-sided and exploratory, and nonsignificant results are not interpreted as equivalence.

The class distribution is strongly imbalanced, with Syntax especially rare. Effective subgroup sizes are consequently small: the held-out static-tool analysis contains only 17 Syntax and 79 Runtime positives, and some self-detection cells have only three cases. Perfect or zero observed rates can therefore have wide intervals. The paper reports denominators and Wilson or task-clustered CIs, emphasizes Macro-F1 and per-class recall alongside Accuracy, and avoids inferential rankings for the descriptive RQ2. These mitigations expose, but do not eliminate, uncertainty caused by rare classes.

Finally, formal paired inference for the Phase-6 and Phase-7 comparisons is restricted to the two hosted models. The retained SLM results are aggregate point estimates and are therefore treated descriptively. This limits conclusions about whether SLM and LLM groups differ in feedback consistency.

5.5 Reliability

The experiments involve multiple stages, including generation, output normalization, execution, knowledge distillation, fine-tuning, static analysis, and iterative tool-assisted interaction. Errors in any stage could propagate to later experiments. To reduce this risk, the study uses the same execution-derived labels across the detection experiments, task-level dataset splits to avoid leakage, deterministic temperature settings during detection and tool-assisted evaluation, and explicit logging of model outputs and tool diagnostics.

5.6 Ethical Considerations and Responsible Use

This computational study uses public programming benchmarks and generated code; it involves no human participants and no personal data. Candidate programs are executed in isolated subprocesses with timeouts rather than on user systems. Hosted-model requests contain public task descriptions and generated benchmark solutions, not private source code.

The proposed detector is not a proof of correctness. False negatives can create unwarranted confidence, while false positives can lead developers to discard valid code. In particular, a passing compilation or absent Pyright diagnostic says little about functional correctness, and passing EvalPlus does not prove correctness for unseen inputs. The approach should therefore complement, not replace, compilation, tests, static/dynamic analysis, security review, and human code review. Model- or tool-generated explanations should be treated as diagnostic suggestions and verified before use in consequential software.

6 Conclusion

This study investigated whether Small Language Models can not only generate code, but also identify hallucinations in generated programs, recognize their own failures, benefit from task specialization, and effectively use external static-analysis evidence. Rather than treating correctness as a binary outcome, we modeled hallucination detection as a multilabel problem involving Syntax, Runtime, Functional, and Correct outcomes, using execution-based evaluation to construct the reference labels. This formulation allowed us to examine code generation and code assessment as related but distinct capabilities.

The results show that strong code generation does not imply strong hallucination detection or self-evaluation. Most evaluated SLMs generated correct solutions for the majority of EvalPlus tasks, but their ability to detect errors varied substantially across models and classes. Functional errors were generally easier to identify, while Runtime errors remained the most persistent challenge. Self-error detection reinforced this separation between generation and evaluation: models that performed well as generators were not necessarily capable of recognizing the errors present in their own outputs. These findings suggest that generation quality alone is insufficient to characterize the reliability of a code-generation model.

Task specialization improves the paired primary metrics for the Accuracy- and Macro-F1-oriented detectors in this setting. Best Acc gains 32.0 Accuracy points (95% CI [25.0, 39.2]) and Weighted MF1 gains 17.2 Macro-F1 points [11.9, 22.3], both after Holm correction. Their class profiles differ: Accuracy-oriented training favors Correct, whereas Macro-F1-oriented training is more balanced. The evaluated balancing strategies improve Accuracy but their Macro-F1 CIs include zero relative to the base. Thus, the training objective changes which errors the detector recognizes, and a detector should be selected for a desired error profile rather than a single aggregate score.

The integration of static-analysis evidence revealed an equally class-dependent behavior. Syntax was the clearest case of successful complementarity: deterministic compilation feedback raised Syntax Recall to almost perfect levels for most detectors and generally reduced syntactic failures during iterative generation. Runtime behaved differently. Pyright provided only limited coverage of the observed Runtime failures, and adding its diagnostics did not consistently improve Runtime classification. However, the same type of feedback more often reduced Runtime failures when models were allowed to revise their code iteratively. This contrast suggests that interpreting evidence for classification and acting on evidence for repair are distinct capabilities. Static feedback also affected classes that the tools do not directly evaluate, particularly the balance between Correct and Functional predictions, showing that models may interpret tool output as broader evidence about program correctness.

The comparison with two substantially larger language models does not reveal a clean separation by scale on these tasks. Several SLM estimates overlap with or exceed the hosted baselines in generation and detection, while both large models show supported gains during iterative tool-assisted

generation. Because size is confounded with model family, training, backend, and precision, these observations neither establish SLM superiority nor isolate a causal scale effect. They show only that compact and specialized models can occupy the same performance range under the evaluated protocols.

Taken together, the findings indicate that reliable code generation should not be approached solely as a problem of increasing model size. Code generation, error recognition, self-evaluation, and correction emerge as distinct capabilities. A model may be strong in one dimension and weak in another, and providing additional evidence does not guarantee that the model will use it effectively. The results therefore support hybrid approaches in which specialized models and deterministic software-engineering tools provide complementary signals rather than relying exclusively on a single general-purpose model.

6.1 Future Work

Future work should first investigate the robustness of these conclusions under broader prompting and sampling conditions. The present study intentionally uses fixed prompts to isolate model, training, and tool effects. Future experiments could evaluate few-shot prompting, explicit reasoning instructions, self-reflection strategies, model-specific prompt optimization, multiple temperatures, fixed seeds, and repeated generations. This would help distinguish stable model capabilities from behaviors that are highly sensitive to prompt formulation or stochastic sampling.

A second direction is to broaden the software-engineering scenarios considered. Replicating the study in languages such as Java, C/C++, Rust, and JavaScript would help determine whether the observed hallucination profiles are language-specific. More importantly, repository-level tasks, multi-file code generation, dependency usage, API interaction, and software repair could reveal failure modes that are largely absent from function-level benchmarks. Such settings would also allow the taxonomy to be extended beyond observable execution outcomes to include dependency, interface, API, and project-level hallucinations.

Future work should also investigate richer forms of external evidence. The current study considers Python compilation and Pyright, but other techniques could provide complementary information, including test generation, symbolic execution, abstract interpretation, data-flow analysis, mutation testing, execution traces, and coverage information. An important question is not only whether a tool detects an error, but whether a language model can correctly interpret and act on the resulting signal. Systematically studying which forms of evidence benefit Syntax, Runtime, and Functional detection could lead to more reliable hybrid verification pipelines.

Finally, the specialized detection pipeline can be expanded in several directions. Future studies could fine-tune additional SLM families, compare rationale-based supervision with label-only training, investigate hierarchical or class-specific detectors, and incorporate confidence calibration or uncertainty estimation. The quality of teacher-generated rationales could also be evaluated directly, allowing the contribution of explanatory supervision to be separated from the effect of the reference labels. More generally, future work could explore whether error detection is better modeled as a single multilabel task or as a staged process that first distinguishes Correct from Incorrect and then identifies the corresponding failure modes.

Overall, the benchmark-conditioned results motivate combining complementary capabilities: compact specialized models for targeted classification, deterministic tools for properties that can be checked directly, and iterative mechanisms that let models act on external evidence. These components reduce particular observed failure modes, but none proves program correctness; deployment still requires testing, review, and safeguards appropriate to the software context.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.

Andriushchenko, G., Garaev, R., Rvanova, L., Shelmanov, A., and Ivanov, V. V. (2026). Efficient hallucination detection in automatic code generation. In Liakata, M., Moreira, V. P., Zhang, J., and Jurgens, D., editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 43197–43220, San Diego, California, United States. Association for Computational Linguistics.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. (2021). Program synthesis with large language models.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code.

Chen, X., Lin, M., Schärli, N., and Zhou, D. (2024). Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*.

DeepSeek-AI (2026). DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Dou, S., Jia, H., Wu, S., Zheng, H., Wu, M., Tao, Y., Zhang, M., Chai, M., Fan, J., Xi, Z., Zheng, R., Wu, Y., Wen, M., Gui, T., Zhang, Q., Qiu, X., and Huang, X. (2025). What's wrong with your code generated by large language models? an extensive study. Accepted by Science China Information Sciences, according to the arXiv record.

Gemma Team, El Abd, S., Aggarwal, V., Algayres, R., Andreev, A., Bachem, O., et al. (2026). Gemma 4 Technical Report.

Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., et al. (2025). Gemma 3 Technical Report.

GLM-4.5 Team, Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., et al. (2025). GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models.

Hsieh, C.-Y., Li, C.-L., Yeh, C.-k., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada. Association for Computational Linguistics.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., et al. (2024). Qwen2.5-Coder Technical Report.

Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. (2022). CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35. Curran Associates, Inc.

Lee, Y., Song, J. Y., Kim, D., Kim, J., Kim, M., and Nam, J. (2025). Hallucination by code generation LLMs: Taxonomy, benchmarks, mitigation, and challenges.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc.

Ludan, J. M., Meng, Y., Nguyen, T., Shah, S., Lyu, Q., Apidianaki, M., and Callison-Burch, C. (2023). Explanation-based finetuning makes models more robust to spurious cues. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4420–4441, Toronto, Canada. Association for Computational Linguistics.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.

Meta Superintelligence Lab (2026). Muse Glimmer 30B Model Card. Hugging Face model card.

Microsoft (2026). Pyright: Static type checker for python. Software repository.

Mistral AI (2025). Introducing Devstral 2 and Mistral Vibe CLI. Official model release.

OpenAI, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., et al. (2025). gpt-oss-120b & gpt-oss-20b model card.

Ornith Team (2026). Ornith-1.5: From self-scaffolding to self-improvement. Official model report.

Pavel, M., Petrovic, N., Mazur, L., Zolfaghari, V., Pan, F., and Knoll, A. (2025). Hallucination in LLM-based code generation: An automotive case study.

Python Software Foundation (2026). Built-in function `compile`. Python 3.13 documentation.

Qwen Team (2025). Qwen3-Coder: Agentic coding in the world. Official model release.

Qwen Team (2026). Qwen3.8-Max: A new bar for coding and cowork. Official model release.

Rastogi, A., Yang, A., Jiang, A. Q., Liu, A. H., Sablayrolles, A., et al. (2025). Devstral: Fine-tuning language models for coding agent applications.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.

Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., and Song, D. (2025). CodeHalu: Investigating code hallucinations in LLMs via execution-based verification. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, pages 25300–25308. AAAI Press.

Xiaomi MiMo Team (2026). MiMo-V2.5. Official model card.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., et al. (2025). Qwen3 Technical Report.

Z.ai (2025). GLM-4.7-Flash model card. Hugging Face model card.